MICRO PROJECT REPORT

BACK-END WEB DEVELOPMENT

SEATSURE: CONCURRENCY-SAFE CAMPUS EVENT REGISTRATION WITH AUTO-PROMOTING WAITLISTS

Submitted by:
Bhavya Sharma
2511021060291

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING (AI AND FUTURE TECHNOLOGIES)

Under the guidance of
Mr. Vaibhav Giri



Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Back-End Web Development** for the micro project titled "SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists" is submitted by Bhavya Sharma (University Registration No. 2511021060291) in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology (Computer Science and Engineering – AI and Future Technologies) of Alliance University. It is a bonafide record of work carried out under my supervision and guidance during the academic year 2026–27. The work presented in this report is original and has not formed the basis for the award of any other degree to the candidate or to anybody else.

……

Mr. Vaibhav Giri



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists** (Back-End Web Development), submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (AI and Future Technologies)** at **Alliance University**, is a record of project work carried out under the supervision of **Mr. Vaibhav Giri**.

The design, implementation and test results described in this report belong to the SeatSure project submitted with it. The course material, documentation and open-source libraries that the work builds on are acknowledged in the references.

This report has not been submitted, in part or in full, for any other degree or award.

Name of the Student	University Registration Number	Signature
Bhavya Sharma	2511021060291	

Acknowledgement

I would like to express my sincere gratitude to everyone who helped me complete this micro project, **SeatSure: Concurrency-Safe Campus Event Registration with Auto-Promoting Waitlists**.

First, I thank my guide, **Mr. Vaibhav Giri**, for the guidance, feedback and encouragement provided throughout the project. I am also grateful to the mentors at Kalvium, whose Back-End Web Development livebook sessions provided the concepts on which this work is built.

I also thank my peers in my Kalvium squad for the discussions and feedback that helped shape the project.

Working on SeatSure taught me that a feature is finished only when it has been shown to work under realistic conditions, with automated tests, measurements and failure scenarios, and not merely when it works once on my own screen.

Finally, I thank my family and friends for their support and patience during the semester.

Thank you.

Bhavya Sharma

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I – Introduction **7**

1.1 Problem Statement 7
1.2 Objectives 8
1.3 Scope of this Report 8
1.4 Theory 8

Chapter II – System Configuration **13**

2.1 System Architecture 13
2.2 Algorithm 16
2.3 Source Code 17
2.4 Connection with Full Stack Web Development and Advanced Database Systems 31
2.5 Setting Up and Running the Project 32
2.6 CRUD over REST: Route Mapping and Validation 32

Chapter III – Results **35**

3.1 Testing Approach 35
3.2 Test Results Summary 35
3.3 Results 35
3.4 Live API Test Cases 40
3.5 Discussion 41

Chapter IV – Conclusion **43**

4.1 Limitations 43
4.2 Future Scope 43

References **45**

Appendix A – Test Cases **46**

Appendix B – Concept Traceability **52**

Abstract

Student clubs at Alliance University run workshops, hackathons and talks in rooms with a fixed number of seats, yet registrations are still collected through Google Forms and WhatsApp, which cannot enforce a seat limit, keep a fair waitlist or report how events fill. This report describes the Back-End Web Development part of **SeatSure**, an integrated micro project that replaces them with a web application. Its core is an Express 5 REST API that decides every registration, so that a seat limit can never be exceeded and a cancelled seat passes automatically to the next student on the waitlist.

The API applies the concepts of Modules 1–3 of the course: REST resources with correct status codes and one JSON envelope; an ordered middleware pipeline (request id, timing, CORS and a 10 KB body limit); express-validator chains that also block mass assignment; `AppError`, `asyncHandler` and a central error handler; and a layered design of routes, thin controllers, services and repositories in which the business rules are pure functions. Registration and cancellation are multi-step workflows that run inside one database transaction which first locks the event row, so concurrent requests can neither overbook a seat nor lose a promotion; transient failures are retried with exponential backoff and jitter. A personal tasks resource shows plain CRUD over REST (201, 200, 204, 400, 404). The API is versioned under `/api/v1`, streams CSV exports, and publishes past-tense events on an EventEmitter bus through a transactional outbox, with timeouts, backoff and graceful degradation when MongoDB is unavailable.

All 71 automated tests pass: 38 unit tests that run the services with in-memory fake repositories, and 33 HTTP integration tests through Supertest against PostgreSQL and MongoDB. Fifty simultaneous registrations for a three-seat event produced exactly three confirmed seats and 47 unique waitlist positions, and a live run against the running server confirmed 10 of 50 students for a 10-seat event with no errors in 414 ms. A simplified version of the workflow without the lock confirmed up to 10 students for 3 seats. With MongoDB stopped, registrations still succeeded and the queued messages were delivered after recovery.

Keywords: Express, REST API, middleware, validation, layered architecture, race conditions, transactions, reliability, event-driven design, transactional outbox

Chapter I – Introduction

Every important decision in SeatSure is made on the server. The React client can make the experience fast, but whether a student receives the last seat or a place on the waitlist, whether a cancelled seat goes to the right person, and whether a request is allowed at all are questions that only the API can answer, because only the API sees every request and owns every transaction.

SeatSure has two kinds of user. **Students** browse upcoming events, register (or join a waitlist when an event is full), cancel, and follow their passes and notifications. **Organisers** of a club create events with their own registration questions, publish them, check attendees in, export attendee lists and read analytics about how their events fill. Every user also keeps a personal to-do list, **My tasks**, with full create, view, update (edit or tick done) and delete operations.

The project is one application split into three folders that match the three subjects of the integrated project: `/frontend` (Full Stack Web Development: the React interface), `/backend` (Back-End Web Development: the Express API) and `/db` (Advanced Database Systems: the PostgreSQL schema, the Prisma data-access layer and MongoDB). Each subject has its own report, and each report is written to be read on its own. This report covers **Back-End Web Development**, whose code lives mainly in `/backend`, and explains how it connects to the other two parts.

1.1 Problem Statement

Student clubs at Alliance University run workshops, hackathons and talks in venues with a fixed number of seats, but registrations are collected through Google Forms and WhatsApp. These tools cannot enforce a seat limit, so popular events get overbooked when many students register at once; waitlists are tracked by hand; seats freed by cancellations are not offered to the next person; and organisers have no data on fill rates or attendance.

SeatSure is a web application where students browse events and register with a hard seat cap that stays correct under simultaneous sign-ups, cancellations automatically promote the next waitlisted student, and organisers manage attendees and view registration analytics.

For the server, the hard part is not storing a registration but deciding it correctly. When many requests for the last seat arrive together, a naive “read the count, then insert” sequence lets several of them see a free seat, and the event is overbooked. Cancellation is just as delicate: the

freed seat must pass to exactly one waitlisted student, even when several people cancel at the same moment. The API must also reject invalid or malicious input, give clients errors they can act on, and keep its core feature working when a secondary service such as MongoDB fails.

1.2 Objectives

- Design a versioned REST API with a consistent response envelope and correct status codes.
- Structure the server in layers so that business rules can be tested without HTTP or a database.
- Validate and sanitise every input at the boundary, and block mass assignment.
- Guarantee that no event is ever overbooked and that waitlist promotion is fair and automatic, even under concurrent requests.
- Make the system reliable: retry transient failures, time out slow dependencies and degrade gracefully.
- Publish domain events so that notifications and the activity feed are decoupled from the registration code.

1.3 Scope of this Report

This report covers Modules 1–3 of the Back-End Web Development course: backend foundations (Module 1), backend application architecture (Module 2) and backend engineering design and principles (Module 3). Lesson numbers such as **M3.6** refer to the lessons of the course livebook. The course keeps data in memory until Module 4; SeatSure stores it in PostgreSQL behind a separate data-access package (`@seatsure/db`), which is the repository layer of this design and is described in detail in the Advanced Database Systems report. Authentication (Module 4) is replaced by a placeholder middleware that reads an `X-User-Id` header and attaches `req.user`, the same contract the course's route chains use. Rate limiting and caching are outside the scope.

1.4 Theory

Table 1.1 lists every Back-End concept applied in SeatSure. The paragraphs after it explain the theory behind each group.

Table 1.1: Back-End concepts applied in SeatSure (Modules 1–3)

Lesson	Concept	Used in SeatSure for
M1.2–1.4	Client–server, HTTP, request–response lifecycle	React client ↔ Express API on port 4000; methods, headers and status-code families
M1.5	REST resource design	Noun resources and sub-resources: `/events`, `/events/:id/registrations`, `/registrations/:id`, `/me/…`
M1.6, M1.15	JSON contract, status codes, error envelope	`{status, data, meta}` and `{status, code, message, errors?, requestId}`; 201 + `Location`, 204, 4xx, 503
M1.7–1.9	Node.js, npm, first Express server	npm workspaces and scripts, `nodemon`, `dotenv`, `app.listen`, catch-all 404 registered last
M1.11	Routing: params vs query, Router modules	One `Router` per resource; `/:id` identifies, `?category=&cursor=` filters
M1.13–1.14	Create, read, update and delete APIs	Tasks: `POST` 201 + `Location`, `GET`, `PATCH`, `DELETE` 204; 404 for a missing id
M2.1	Node.js runtime and event loop	Every `await` is a point where another request can run, which is why races happen
M2.2	Streams	Attendee CSV export streamed with `Readable.from` and `pipeline` (back-pressure)
M2.4	Middleware pipeline	request id → timer → CORS → JSON (10 KB) → routes → 404 → error handler
M2.5	Request validation	express-validator chains; one 400 response that lists every invalid field
M2.6	API hardening	`matchedData` allow-list against mass assignment, payload limit, answer sanitisation, header validation
M2.7	Error-handling architecture	`AppError`, `asyncHandler`, one four-argument error handler registered last
M2.9–2.11	Layers, thin controllers, services, repository pattern	routes → controllers → services → `@seatsure/db` repositories; in-memory fakes in unit tests
M2.12	Supporting layers and configuration	`config/index.js` (environment-driven, frozen), `utils/`, `validators/`
M3.1, M3.3	Data flow and engineering principles	PostgreSQL as the single source of truth, explicit trust boundaries, defensive programming
M3.2	Domain rules and workflows	Event state machine; register; cancel → promote; capacity increase → promote
M3.4–3.6	Concurrency, race conditions, transactions	One transaction per workflow with a row lock; 50 concurrent requests in a test
M3.8–3.9	API design, pagination, filtering, sorting	Cursor (`nextCursor`, `hasMore`) and offset (`page`, `totalPages`) metadata; whitelisted filters and sorts
M3.10	API versioning	`/api/v1`; deprecated `/api/events` alias with `Deprecation`, `Sunset` and `Link` headers
M3.11	Reliability engineering	Retries with exponential backoff and jitter, timeouts, graceful degradation when MongoDB fails
M3.12	Event-driven systems	EventEmitter bus, past-tense events, an outbox dispatcher and two subscribers

1.4.1 Backend Foundations (Module 1)

HTTP and REST (M1.2–M1.5). A client sends an HTTP request (method, path, headers and an optional body) and the server answers with a status code, headers and a body. HTTP is stateless, so each request must carry everything the server needs, including who is asking. In a

REST design the URL names a resource with a noun (`/events/42`), the method says what to do with it (GET reads, POST creates, PATCH updates, DELETE removes), and related resources are nested (`/events/42/registrations`).

Contracts and status codes (M1.6, M1.15). Clients can only be simple if the server is predictable. Every response therefore uses one envelope, and the status code tells the client what happened: 200 or 201 (with a `Location` header) for success, 204 for an empty success, 400 for malformed input, 401 or 403 for identity and permission problems, 404 for a missing resource, 409 for a conflict with the current state, 422 for input that is well-formed but breaks a rule, and 503 when a dependency is unavailable. Internal details such as stack traces or SQL must never reach the client.

Node.js and Express (M1.7–M1.11). Express builds a server from small functions. `express.Router` groups the routes of one resource; route parameters (`/:id`) identify a resource while query strings (`?category=TALK`) filter a collection; `express.json()` fills `req.body`; and a catch-all handler registered after all routes answers 404 for unknown paths.

1.4.2 Backend Application Architecture (Module 2)

The runtime (M2.1, M2.2). Node.js runs JavaScript on one thread and handles many requests by never blocking: whenever a handler waits (`await`) for the database, the event loop runs other requests. This is efficient, but it also means that two requests can interleave at every `await`, which is the root cause of the race conditions discussed in Module 3. Streams move large data in chunks rather than loading it all into memory, and `pipeline` applies back-pressure when the receiver is slower than the sender.

Middleware, validation and hardening (M2.4–M2.6). Middleware functions form an ordered pipeline; each one can modify the request, end it early or pass it on with `next()`, so the order matters. Validation checks that input has the right shape and type; sanitisation cleans it (for example trimming text). The boundary between the client and the server is a *trust boundary*: nothing that crosses it is trusted. Two common attacks are mass assignment, in which a client sends fields it should not control (such as `status: "CONFIRMED"`), and oversized payloads. An allow-list of accepted fields and a body-size limit stop both.

Error handling (M2.7). Expected failures (*operational errors*) are thrown as `AppError(message, statusCode)`. `asyncHandler` forwards rejected promises to `next(err)`, and one error handler with four arguments, registered last, turns every error into

the standard envelope. Unexpected failures (*programmer errors*) are logged in full but shown to the client only as a generic 500.

Layers (M2.8–M2.12). As a server grows, putting everything in one file creates hidden coupling and slow, fragile tests. A layered design gives each folder one job: `routes/` wire URLs to handlers, `controllers/` extract input and format output, `services/` apply the business rules, and repositories store data. Imports flow in one direction only. Because services receive plain values and never see `req` or `res`, and repositories hide the storage, a service can be tested with fake repositories and the database could be replaced without touching any rule. Configuration, validators and shared utilities live in their own supporting folders.

1.4.3 Backend Engineering Design and Principles (Module 3)

Data flow, rules and principles (M3.1–M3.3). A request flows through middleware, controller, service and repository to the database and back. Business rules ("registration closes when the event starts") belong in the service layer as checks that throw `AppError`, and a multi-step operation is modelled as an ordered *workflow*. Engineering principles guide the design: a single source of truth for each fact, explicit trust boundaries, defensive programming and single responsibility.

Concurrency and transactions (M3.4–M3.6). Concurrent requests share state. A *race condition* occurs when the result depends on the timing of their steps: two requests read "39 of 40 seats taken", both decide a seat is free, and both insert, which is an inventory oversell. Other forms are the lost update and duplicate processing. They are dangerous because they rarely appear in manual testing. A transaction groups several writes so that they succeed or fail together (atomicity), and with an appropriate lock or isolation level it also closes the window between "check" and "act".

API design and versioning (M3.8–M3.10). List endpoints return pagination metadata: offset pagination for numbered pages, cursor pagination for feeds that grow while they are read. Filters and sort options are whitelisted. When an API must change in a breaking way, a version prefix (`/api/v1`) lets old clients keep working, and deprecated paths announce their replacement before they are removed.

Reliability and events (M3.11, M3.12). Networks and dependencies fail. A transient failure can be retried with *exponential backoff* (waiting longer after each attempt) and *jitter* (a random component, so that many clients do not retry at the same instant); a slow dependency is bounded by a *timeout*; and when a non-essential dependency is down, the system should *degrade*

gracefully instead of failing completely. In an event-driven design, a producer announces a past-tense fact ("registration.confirmed") on an event bus without knowing who listens; subscribers react independently. Node's `EventEmitter` provides such a bus inside one process.

Chapter II – System Configuration

The API was developed and tested on a single laptop. Table 2.1 lists the hardware and software.

Table 2.1: Hardware and software configuration

Component	Configuration
Processor and memory	Apple M4 (10 cores), 16 GB RAM
Operating system	macOS 26.7 (arm64)
Runtime	Node.js 22.23.2, npm 10.9.8 (npm workspaces: `frontend`, `backend`, `db`)
Web framework	Express 5.3.0 (JavaScript ES modules), `cors` 2.8.6, `dotenv` 17.4.2
Validation	express-validator 7.3.2
Development tools	`nodemon` 3.1.14 (restarts on change); `tsx` 4.23.15 runs the server so it can import the TypeScript data-access package
Data access (repository layer)	`@seatsure/db`: Prisma ORM 7.10.0 over PostgreSQL 18.4; Mongoose 9.11.4 over MongoDB 8.2 (in-memory server in development and tests)
Testing	Vitest 5.0.5, Supertest 7.3.1 (HTTP integration tests)
Manual testing	`curl`, the React client and the browser's Network tab

2.1 System Architecture

Figure 2.1 shows where the API sits. It is the only component that talks to both databases: PostgreSQL holds every fact that matters (users, events, registrations, the outbox), and MongoDB holds data derived from those facts (notifications and an activity feed), which is written asynchronously by event subscribers.

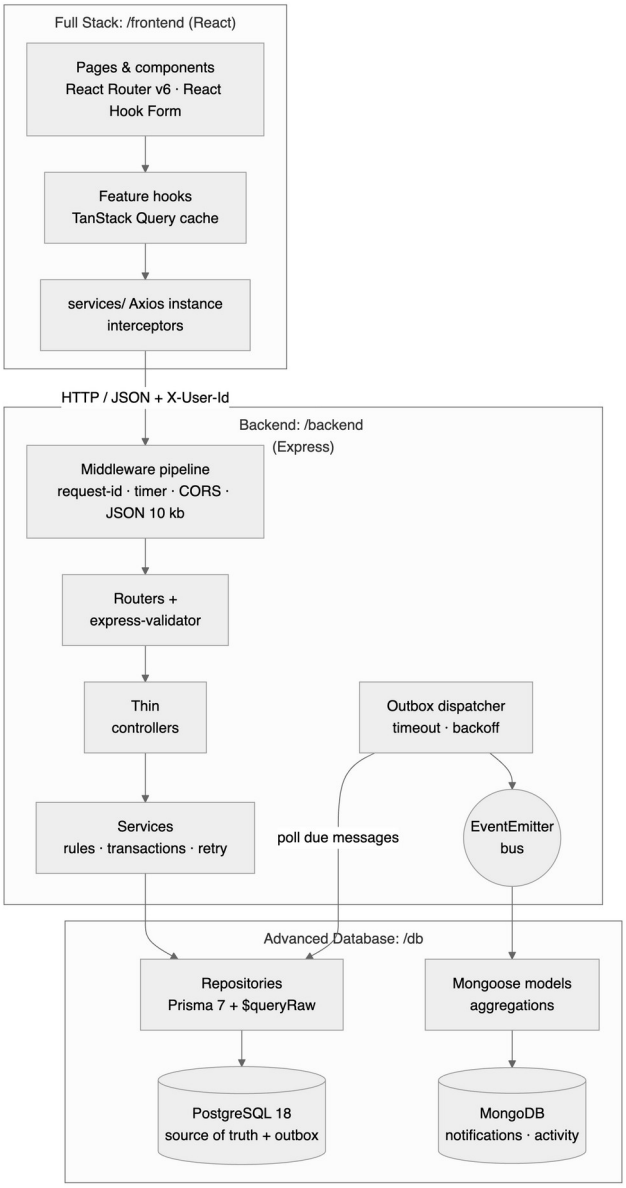


Figure 2.1: SeatSure architecture: client, API and data layers

Inside `/backend`, each folder is one layer (Listing 2.1). Dependencies flow downwards only: routes know controllers, controllers know services, and services know repositories through the composition root `container.js`.

Listing 2.1: Layered folder structure of `backend/src`

```
backend/src/
├── app.js          middleware pipeline (order matters) + routers + 404 + error
    handler
├── server.js       starts the HTTP server, the outbox dispatcher, graceful shutdown
├── container.js    composition root: wires repositories, bus, dispatcher, services
├── config/         environment-driven, frozen configuration
├── routes/         one Router per resource; /api/v1 + deprecated /api alias
├── validators/     express-validator chains
├── controllers/    thin: extract → call one service → format the response
├── services/       business rules and workflows (rules/eventRules.js: pure
    functions)
├── middleware/     requestId, requestTimer, auth (demo identity), validate,
                    notFound, errorHandler, deprecation
├── events/         EventEmitter bus, outbox dispatcher, MongoDB subscribers
└── utils/          AppError, asyncHandler, retry/withTimeout/backoff, respond,
    logger
```

The API exposes the resources in Table 2.2. Requests that need a user send the `X-User-Id` header.

Table 2.2: REST API (all JSON, under `/api/v1`)

Method	Path	Who	Purpose
GET	`/health`	anyone	PostgreSQL/MongoDB status, outbox backlog, seat-counter integrity
GET	`/users`	anyone	Demo identities
GET	`/me` · `/me/registrations` · `/me/notifications`	user	Profile, passes (with waitlist positions), notifications
PATCH	`/me/notifications` `{read:true}`	user	Mark all notifications read
GET	`/clubs` · `/venues`	anyone	Reference data
GET	`/events?q&clubId&category&availability&sort&cursor&limit`	anyone	Catalogue (cursor pagination)
GET	`/events/:id`	anyone	Event detail (+ my registration)
GET	`/events/mine?page&pageSize&status`	organiser	Organiser dashboard (offset pagination)
POST · PATCH · DELETE	`/events` · `/events/:id`	organiser of the club	Create draft · edit/publish/cancel/capacity · delete draft
POST	`/events/:id/registrations` `{answers}`	user	Register: 201 `{outcome: CONFIRMED | WAITLISTED, position}`
GET	`/events/:id/registrations?page&pageSize&status&search&answers[key]=value`	organiser	Attendees (offset pagination + JSONB filter)
GET	`/events/:id/registrations/export`	organiser	Streamed CSV
GET	`/events/:id/activity`	anyone	MongoDB activity feed (`meta.degraded` if MongoDB is down)
DELETE	`/registrations/:id`	owner or organiser	Cancel (+ automatic promotion)
PATCH	`/registrations/:id` `{checkedIn:true}`	organiser	Check in (idempotent)
GET	`/tasks`	user	My tasks, open first, with `meta {total, open, done}`
POST	`/tasks` `{title, notes?, dueDate?}`	user	Create a task: 201 + `Location` (empty title: 400)

Method	Path	Who	Purpose
GET · PATCH · DELETE	`/tasks/:id`	owner	Read · edit or tick done `{title?, notes?, dueDate?, completed?}` · delete (204). Another user's or a missing task: 404
GET	`/analytics/overview?scope=mine|all` · `/analytics/activity`	organiser	Window-function reports · MongoDB aggregations
POST	`/dev/mongo/outage` · `/dev/mongo/restore`	organiser, dev only	Graceful-degradation demo
GET	`/api/events`	anyone	**Deprecated** alias (sends `Deprecation`, `Sunset` and `Link` headers)

2.2 Algorithm

The three central workflows of the API are written below as algorithms; the lesson that each step applies is given in brackets.

Algorithm 1: Registering for an event (POST /api/v1/events/:id/registrations)

1. Middleware assigns a request id, starts a timer, checks CORS and parses JSON up to 10 KB (M2.4, M2.6).
2. `requireUser` validates the `X-User-Id` header and loads the user (400 if malformed, 401 if missing or unknown) (M3.3).
3. `registerRules` and `validate` check that the id is a positive integer and `answers` is a plain object with at most 10 keys; every other body field is dropped (M2.5, M2.6).
4. The controller calls `registrationService.register(user, eventId, answers)` (M2.10).
5. BEGIN a transaction and lock the event row with `SELECT … FOR UPDATE`; concurrent requests for the same event wait here (M3.6).
6. Check the rules: the event is published and registration is still open; the student has no active registration; the answers match the event's questions (M3.2).
7. If `confirmed_count < capacity`, insert a CONFIRMED registration and add one to the counter; otherwise insert a WAITLISTED registration and compute its position.
8. Insert an outbox message (`registration.confirmed` or `registration.waitlisted`) and COMMIT, which releases the lock (M3.12).
9. If the transaction failed because of a deadlock, a serialisation failure or a pool timeout, repeat from step 5 after an exponential backoff, at most three times (M3.11).
10. Respond 201 with `{outcome, position, seatsLeft}` and a `Location` header, then wake the outbox dispatcher (M1.15).

Algorithm 2: Cancelling with automatic promotion (DELETE /api/v1/registrations/:id)

1. Load the registration (404 if it does not exist) and note whether the caller owns it.
2. BEGIN and lock the event row; a caller who is not the owner must organise the event's club (403 otherwise).
3. Refuse if the event has started (409) or, re-reading under the lock, if the registration is already cancelled (409).
4. Mark the registration CANCELLED and write a `registration.cancelled` message.
5. If it was CONFIRMED, take the oldest WAITLISTED registration. If there is one, promote it to CONFIRMED (the counter does not change, because the seat passes straight on) and write `registration.promoted`; otherwise subtract one from the counter.
6. COMMIT and respond 200 with the cancelled registration and the promoted student, if any.

Algorithm 3: Delivering outbox messages (dispatcher)

1. Every 2 seconds, or immediately after a commit, fetch up to 100 unpublished messages whose `available_at` time has passed, oldest first.
2. Publish each message on the EventEmitter bus with its UUID `outboxId`, bounded by a 5-second timeout (M3.11, M3.12).
3. Subscribers write a notification and an activity document to MongoDB; a duplicate `outboxId` means the message was already delivered and counts as success.
4. On success, mark the message published. On failure, increase `attempts`, postpone `available_at` by 2 s, 4 s, 8 s … (at most 60 s) and stop the round.
5. Rounds never overlap: they run one after another on a promise chain, so no message is delivered twice in parallel.

2.3 Source Code

This section shows how and where each concept is implemented. Every listing is copied from the project, with its file path and line numbers; lines marked "omitted" were left out for brevity. Comments carry tags such as `[BACKEND M3.6]`, so `grep -rn "BACKEND M" backend/src` lists every place a concept is used, and Appendix B maps each concept to its files and tests.

2.3.1 The middleware pipeline (M2.4, M2.6, M1.9)

`createApp` builds the Express application without opening a port, so tests can create it with any dependencies. The order of the pipeline is deliberate: the request id comes first so that every later log line carries it, the body limit is enforced before any route runs, the 404 handler follows all routes, and the error handler is last (Listing 2.2).

Listing 2.2: Ordered middleware pipeline (`backend/src/app.js`, lines 35–57)

```
35  // [BACKEND M2.4: Ordered middleware pipeline]
36  app.use(requestId); // 1. tag the request (must be first so every later log line
        has the id)
37  app.use(requestTimer); // 2. time + log it
38  app.use(
39    // 3. [FULLSTACK M2.1: CORS] only the React dev/prod origins may call the API
          from a browser
40    cors({
41      origin: config.corsOrigins,
42      exposedHeaders: ['X-Request-Id', 'X-Response-Time', 'Deprecation', 'Sunset',
            'Link', 'Location'],
43    }),
44  );
45  app.use(express.json({ limit: config.bodyLimit })); // 4. parse JSON, reject
        bodies over 10 kb (413)
46
47  // Production: serve the built React app. Any GET outside /api returns
        index.html, so React Router can
48  // handle deep links such as /events/12; API paths still reach the routers and
        the JSON 404 handler.
49  if (config.clientDist && existsSync(path.join(config.clientDist, 'index.html')))
        {
50    app.use(express.static(config.clientDist, { index: false, maxAge: '1h' }));
51    app.get(/^\/(?!api(?:\/|$)).*/, (req, res) =>
          res.sendFile(path.join(config.clientDist, 'index.html')));
52  }
53  app.get('/', (req, res) => ok(res, { name: 'SeatSure API', version: 'v1',
        health: '/api/v1/health' }));
54  app.use('/api', createApiRouter({ controllers, auth, config, mongoControl:
        container.mongoControl })); // 5. routes
55
56  app.use(notFound); // 6. nothing matched -> 404
57  app.use(errorHandler); // 7. LAST: every error becomes one consistent envelope
```

The request-id middleware reuses an id sent by the caller only if it looks like an id; anything else (for example an injected `<script>`) is replaced. The id is echoed in the `X-Request-Id` header and in every error body, so a failing request can be traced from the browser's Network tab to the server log (Listing 2.3).

Listing 2.3: Request id with a trust-boundary check (`backend/src/middleware/requestId.js`, lines 6–14)

```
 6  const SAFE_ID = /^[A-Za-z0-9._-]{8,64}$/;
 7
 8  export function requestId(req, res, next) {
 9    // Trust boundary ([BACKEND M3.3]): reuse a caller-supplied id only if it
          looks like an id.
10    const incoming = req.get('X-Request-Id');
11    req.id = incoming && SAFE_ID.test(incoming) ? incoming : randomUUID();
12    res.setHeader('X-Request-Id', req.id);
13    next();
14  }
```

Identity is a placeholder for Module 4, but it is still treated as untrusted input: the header must be a positive integer and must name an existing user (Listing 2.4).

Listing 2.4: Demo identity middleware (placeholder for real authentication) (`backend/src/middleware/auth.js`, lines 7–15)

```
 7  async function loadUser(req) {
 8    const raw = req.get('X-User-Id');
 9    if (!raw) return null;
10    // Trust boundary: the header is client input, so validate it like any other
          input.
11    if (!/^\d{1,9}$/.test(raw)) throw new AppError('X-User-Id must be a positive
          integer', 400, 'BAD_USER_HEADER');
12    const user = await users.findWithClubs(Number(raw));
13    if (!user) throw new AppError('Unknown user. Choose a demo user again.', 401,
          'UNKNOWN_USER');
14    return user;
15  }
```

2.3.2 REST routing and versioning (M1.5, M1.11, M3.10)

Each resource has its own router, mounted under `/api/v1`. The old unversioned catalogue path `/api/events` still works, but it is marked as deprecated with standard headers that name its successor and the date on which it will be removed (Listing 2.5, Listing 2.6).

Listing 2.5: Versioned API router with a deprecated alias (`backend/src/routes/index.js`, lines 16–37)

```
16  const v1 = express.Router();
17  v1.get('/health', controllers.misc.health);
18  v1.get('/users', controllers.misc.demoUsers);
19  v1.get('/clubs', controllers.misc.clubs);
20  v1.get('/venues', controllers.misc.venues);
21  v1.use('/events', createEventsRouter({ events: controllers.events,
        registrations: controllers.registrations, auth }));
22  v1.use('/registrations', createRegistrationsRouter({ registrations:
        controllers.registrations, auth }));
23  v1.use('/me', createMeRouter({ misc: controllers.misc, registrations:
        controllers.registrations, auth }));
24  v1.use('/analytics', createAnalyticsRouter({ misc: controllers.misc, auth }));
25  v1.use('/tasks', createTasksRouter({ tasks: controllers.tasks, auth }));
26  if (config.enableDevRoutes) v1.use('/dev', createDevRouter({ auth, mongoControl
        }));
27
28  const api = express.Router();
29  api.use('/v1', v1);
30  api.get(
31    '/events',
32    deprecated({ successor: '/api/v1/events', sunset: 'Tue, 30 Jun 2027 00:00:00
          GMT' }),
33    listEventsRules,
34    validate,
35    controllers.events.list,
36  );
37  return api;
```

Listing 2.6: Deprecation headers (`backend/src/middleware/deprecation.js`, lines 4–9)

```
4  export const deprecated = ({ successor, sunset }) => (req, res, next) => {
5    res.setHeader('Deprecation', 'true');
6    res.setHeader('Sunset', sunset);
7    res.setHeader('Link', `<${successor}>; rel="successor-version"`);
8    next();
9  };
```

A route is a chain of small functions: identity, role, validation and finally the controller. Cancelling uses DELETE and returns 200 with a body (not 204), because the client needs to know who was promoted (Listing 2.7).

Listing 2.7: Registration routes as middleware chains (`backend/src/routes/registrations.routes.js`, lines 7–12)

```
7  const router = express.Router();
8  // DELETE = cancel. 200 with a body (not 204) because the client needs to know
       who got promoted.
9  router.delete('/:id', auth.requireUser, registrationIdRules, validate,
       registrations.cancel);
10 // PATCH { checkedIn: true }: the only field an organiser may change on a
       registration.
11 router.patch('/:id', auth.requireUser, requireRole('ORGANISER'), checkInRules,
       validate, registrations.checkIn);
12 return router;
```

2.3.3 Validation and mass-assignment protection (M2.5, M2.6)

After an express-validator chain, `validate` either stops the request with one 400 response that lists every invalid field, or exposes *only* the declared fields as `req.valid`. A client that sends `status: "CONFIRMED"` or `confirmedCount: 999` is ignored, because no validator declares those fields (BE-I08, BE-I13) (Listing 2.8).

Listing 2.8: Validation result and allow-list (matchedData) (`backend/src/middleware/validate.js`, lines 9 –25)

```
9  export function validate(req, res, next) {
10   const result = validationResult(req);
11   if (!result.isEmpty()) {
12     const errors = result.array({ onlyFirstError: true }).map((e) => ({
13       field: e.type === 'field' ? e.path : e.type,
14       message: e.msg,
15       location: e.type === 'field' ? e.location : undefined,
16     }));
17     return next(new AppError('Some fields are invalid', 400, 'VALIDATION_ERROR',
           errors));
18   }
19   req.valid = {
20     params: matchedData(req, { locations: ['params'] }),
21     query: matchedData(req, { locations: ['query'] }),
22     body: matchedData(req, { locations: ['body'], includeOptionals: false }),
23   };
24   next();
25 }
```

Listing 2.9: The register body accepts only an answers object (`backend/src/validators/registrations.validators.js`, lines 5–13)

```
 5  const plainObject = (value) => {
 6    if (value === null || typeof value !== 'object' || Array.isArray(value)) throw
          new Error('answers must be an object');
 7    if (Object.keys(value).length > 10) throw new Error('Too many answers');
 8    return true;
 9  };
10
11  // POST /events/:id/registrations: only `answers` is accepted. A client cannot
       choose its own
12  // status ("CONFIRMED") or event; those come from the URL and the server's seat
       decision.
13  export const registerRules = [idParam('id', 'Event id'),
       body('answers').default({}).custom(plainObject)];
```

2.3.4 Errors: AppError, asyncHandler and the central handler (M2.7, M1.15)

Services throw `AppError` with a status code and a stable machine-readable code; they never call `res.status()` themselves (Listing 2.10). `asyncHandler` sends any rejected promise to `next(err)`, so handlers need no `try/catch`:

Listing 2.10: Operational error class (`backend/src/utils/AppError.js`, lines 4–21)

```
 4  export class AppError extends Error {
      // … 6 lines omitted …
11    constructor(message, statusCode = 400, code = undefined, errors = undefined) {
12      super(message);
13      this.name = 'AppError';
14      this.statusCode = statusCode;
15      this.status = String(statusCode).startsWith('4') ? 'fail' : 'error';
16      this.code = code ?? defaultCode(statusCode);
17      this.errors = errors;
18      this.isOperational = true;
19      Error.captureStackTrace?.(this, this.constructor);
20    }
21  }
```

Listing 2.11: Forwarding async errors (`backend/src/utils/asyncHandler.js`, line 5)

```
 5  export const asyncHandler = (fn) => (req, res, next) => Promise.resolve(fn(req,
       res, next)).catch(next);
```

The central handler normalises every error (AppError, body-parser errors, timeouts and database errors translated from Prisma) into the standard envelope. Anything it does not recognise is a programmer error: it is logged in full with the request id and answered with a generic 500, so stack traces and SQL never leak (BE-U29) (Listing 2.12).

Listing 2.12: Central error handler (`backend/src/middleware/errorHandler.js`, lines 54–82)

```
54  export function errorHandler(err, req, res, next) {
55    const appError = normalise(err);
56
      // … 5 lines omitted …
62
63    if (appError) {
64      if (appError.statusCode >= 500) logger.error(appError.message, { id: req.id,
            code: appError.code, cause: err?.message });
65      return res.status(appError.statusCode).json({
66        status: appError.status,
67        code: appError.code,
68        message: appError.message,
69        ...(appError.errors ? { errors: appError.errors } : {}),
70        requestId: req.id,
71      });
72    }
73
74    // Programmer error: full details in the server log only.
75    logger.error('Unhandled error', { id: req.id, name: err?.name, message:
          err?.message, stack: err?.stack });
76    return res.status(500).json({
77      status: 'error',
78      code: 'INTERNAL_ERROR',
79      message: 'Something went wrong. Please try again later.',
80      requestId: req.id,
81    });
82  }
```

2.3.5 Layers: thin controllers and the repository pattern (M2.9–M2.12)

Controllers only extract validated input, call one service method and format the response (Listing 2.13). Pagination metadata is added here, because it belongs to the HTTP contract rather than to the business rules (M3.8).

Listing 2.13: Thin controllers (`backend/src/controllers/events.controller.js`, lines 8–44)

```
 8  list: asyncHandler(async (req, res) => {
 9    const filters = req.valid.query;
10    const limit = filters.limit ?? config.pagination.defaultLimit;
11    const { items, nextCursor } = await eventService.list({ ...filters, limit });
12    // [BACKEND M3.8: Pagination metadata] Clients follow nextCursor; null means
          "last page".
13    ok(res, items, { limit, nextCursor, hasMore: nextCursor !== null });
14  }),
15
    // … 25 lines omitted …
41  register: asyncHandler(async (req, res) => {
42    const result = await registrationService.register(req.user,
          req.valid.params.id, req.valid.body.answers);
43    created(res, result, `/api/v1/registrations/${result.registration.id}`);
44  }),
```

The composition root is the only file that knows which concrete repositories the services use (Listing 2.14). Unit tests build the same services with in-memory fakes that implement the same functions, such as `lockForUpdate` and `changeConfirmedCount` (Listing 2.15). The services cannot tell the difference, which shows that the storage is swappable.

Listing 2.14: Composition root: services receive repositories (`backend/src/container.js`, lines 44–51)

```
44  const shared = {
45    events: eventRepository,
46    registrations: registrationRepository,
47    catalog: catalogRepository,
48    outbox: outboxRepository,
49    runInTransaction: (work) => withTransaction(work),
50    afterCommit: () => dispatcher.kick(), // deliver new outbox messages promptly
51  };
```

Listing 2.15: An in-memory fake repository used by unit tests (`backend/tests/helpers/fakes.js`, lines 18–26)

```
18  async lockForUpdate(_tx, id) {
19    return clone(state.events.get(id) ?? null);
20  },
21  async changeConfirmedCount(_tx, id, delta) {
22    const e = state.events.get(id);
23    e.confirmedCount += delta;
24    if (e.confirmedCount > e.capacity) throw Object.assign(new Error('check
          violation'), { kind: 'CHECK_VIOLATION' });
25    return clone(e);
26  },
```

2.3.6 Domain rules and workflows (M3.2)

The business rules are pure functions in `services/rules/eventRules.js`: they take plain values and either return or throw `AppError`, so each rule is unit-tested in isolation (BE-U01–U08). The event lifecycle is a small state machine (Listing 2.16).

Listing 2.16: Event lifecycle as a state machine (`backend/src/services/rules/eventRules.js`, lines 9–20)

```
 9  export const STATUS_TRANSITIONS = Object.freeze({
10    DRAFT: ['PUBLISHED'],
11    PUBLISHED: ['CANCELLED'],
12    CANCELLED: [],
13  });
14
15  export function assertTransition(from, to) {
16    if (from === to) return;
17    if (!STATUS_TRANSITIONS[from]?.includes(to)) {
18      throw new AppError(`An event cannot go from ${from} to ${to}`, 409,
            'INVALID_STATUS_TRANSITION');
19    }
20  }
```

Listing 2.17: Registration rules and the seat decision (`backend/src/services/rules/eventRules.js`, lines 29–56)

```
29  export function assertOpenForRegistration(event, now) {
30    if (event.status !== 'PUBLISHED') {
31      throw new AppError('Registrations are not open for this event', 409,
            'REGISTRATION_NOT_OPEN');
32    }
33    const closesAt = event.registrationClosesAt ?? event.startsAt;
34    if (now >= new Date(closesAt)) {
35      throw new AppError('Registration for this event has closed', 409,
            'REGISTRATION_CLOSED');
36    }
37  }
38
    // … 13 lines omitted …
52
53  /** The seat decision. Callers must hold the event row lock, or two requests
        could both see a free seat. */
54  export function hasFreeSeat(event) {
55    return event.confirmedCount < event.capacity;
56  }
```

2.3.7 Concurrency: one transaction around the whole workflow (M3.4–M3.6, M3.11)

The comment at the top of `registrationService.js` describes the race being prevented. Request A reads "39 of 40 confirmed" and request B reads the same; both insert a CONFIRMED row, and the event ends at 41 of 40. The fix is to run the whole read–check–write sequence in one transaction that first locks the event row. B then waits at the lock until A commits, sees 40 of 40 and is waitlisted. Transient failures are retried (Listing 2.18); business-rule failures are not (BE-U13, BE-U14).

Listing 2.18: Retrying only transient transaction failures (`backend/src/services/registrationService.js`, lines 26–36)

```
26  const isTransient = (err) => err?.kind === 'WRITE_CONFLICT' || err?.kind ===
        'TIMEOUT';
27
28  export function createRegistrationService({ events, registrations, outbox,
        runInTransaction, afterCommit = () => {}, clock = () => new Date() }) {
29    // [BACKEND M3.11: Reliability] A transaction that lost a
          deadlock/serialisation race or timed out
30    // waiting for a pooled connection is re-run with exponential backoff +
          jitter.
31    const transactWithRetry = (work) =>
32      retry(() => runInTransaction(work), {
33        retries: 3,
34        shouldRetry: isTransient,
35        onRetry: (err, attempt, delay) => logger.warn('Retrying registration
              transaction', { attempt, delay, kind: err.kind }),
36      });
```

Listing 2.19: The register workflow inside one locked transaction (`backend/src/services/registrationService.js`, lines 53–90)

```
53  async register(user, eventId, rawAnswers) {
54    const result = await transactWithRetry(async (tx) => {
55      const event = await events.lockForUpdate(tx, eventId);
56      if (!event) throw new AppError(`Event ${eventId} not found`, 404,
            'EVENT_NOT_FOUND');
57      const now = clock();
58      assertOpenForRegistration(event, now);
59
60      if (await registrations.findActive(eventId, user.id, tx)) {
61        throw new AppError('You are already registered for this event', 409,
              'ALREADY_REGISTERED');
62      }
63      const answers = normaliseAnswers(event.questions, rawAnswers);
64
65      const confirmed = hasFreeSeat(event);
66      const registration = await registrations.create(tx, {
67        eventId,
68        userId: user.id,
69        status: confirmed ? 'CONFIRMED' : 'WAITLISTED',
70        answers,
71      });
72      if (confirmed) await events.changeConfirmedCount(tx, eventId, +1);
73
74      const position = confirmed ? null : (await
              registrations.waitlistPositions([registration.id],
              tx)).get(registration.id) ?? null;
75      await outbox.add(
76        tx,
77        confirmed ? OUTBOX.REGISTRATION_CONFIRMED :
              OUTBOX.REGISTRATION_WAITLISTED,
78        registrationMessage(eventRef(event), { id: registration.id, userId:
              user.id, userName: user.name }, now, position ? { position } : {}),
79      );
80
81      return {
          // … 4 lines omitted …
86      };
87    });
88    afterCommit();
89    return result;
90  },
```

2.3.8 Cancel and promote in the same transaction (M3.2, M3.6)

Cancellation and promotion form one workflow. Because they commit together under the same row lock, a freed seat can neither be lost (cancelled but nobody promoted) nor given twice (two cancellations promoting the same student). The registration is re-read under the lock, because a parallel request may have changed it in the meantime (BE-I23) (Listing 2.20).

Listing 2.20: Atomic cancel-and-promote workflow (`backend/src/services/registrationService.js`, lines 100–125)

```
100  const result = await transactWithRetry(async (tx) => {
101    const event = await events.lockForUpdate(tx, existing.eventId);
102    if (!isOwner) assertOrganiserOf(user, event.clubId);
103    const now = clock();
104    assertCanCancel(event, now);
105
106    // Re-read under the lock: a parallel request may have changed it since we
           looked.
107    const reg = await registrations.findById(registrationId, tx);
108    if (reg.status === 'CANCELLED') throw new AppError('This registration is
           already cancelled', 409, 'ALREADY_CANCELLED');
109
110    await registrations.cancel(tx, reg.id, now);
111    const ref = eventRef(event);
112    await outbox.add(tx, OUTBOX.REGISTRATION_CANCELLED, registrationMessage(ref, {
           id: reg.id, userId: reg.userId, userName: reg.user.name }, now, {
           byOrganiser: !isOwner }));
113
114    let promoted = null;
115    if (reg.status === 'CONFIRMED') {
116      const [next] = await registrations.nextWaitlisted(tx, event.id, 1);
117      if (next) {
118        await registrations.promote(tx, [next.id], now); // seat passes straight
               on: count unchanged
119        await outbox.add(tx, OUTBOX.REGISTRATION_PROMOTED,
               registrationMessage(ref, next, now));
120        promoted = { registrationId: next.id, userId: next.userId, userName:
               next.userName };
121      } else {
122        await events.changeConfirmedCount(tx, event.id, -1);
123      }
124    }
125    return { cancelled: { id: reg.id, previousStatus: reg.status, cancelledAt: now
           }, promoted };
```

2.3.9 Reliability: retry, backoff, timeouts and graceful degradation (M3.11)

`retry` doubles the delay after every failed attempt, caps it, and randomises it ("jitter") so that clients that failed together do not retry together. `withTimeout` bounds a slow dependency, and `backoffMs` gives the outbox schedule of 2 s, 4 s, 8 s … 60 s (Listing 2.21).

Listing 2.21: Retry with exponential backoff and jitter, timeouts (`backend/src/utils/retry.js`, lines 10–42)

```
10  export async function retry(operation, { retries = 3, baseMs = 40, maxMs =
        1_000, shouldRetry = () => true, onRetry } = {}) {
11    for (let attempt = 0; ; attempt++) {
12      try {
13        return await operation(attempt);
14      } catch (err) {
15        if (attempt >= retries || !shouldRetry(err)) throw err;
16        const delay = Math.round(Math.min(maxMs, baseMs * 2 ** attempt) * (0.5 +
              Math.random() / 2));
17        onRetry?.(err, attempt + 1, delay);
18        await sleep(delay);
19      }
20    }
21  }
22
    // … 7 lines omitted …
30  /** Rejects if `promise` has not settled within `ms`, so a slow dependency
        cannot hang a request. */
31  export function withTimeout(promise, ms, label = 'operation') {
32    let timer;
33    const timeout = new Promise((_, reject) => {
34      timer = setTimeout(() => reject(new TimeoutError(label, ms)), ms);
35    });
36    return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
37  }
38
39  /** Exponential backoff used by the outbox dispatcher between delivery attempts
        (2 s .. 60 s). */
40  export function backoffMs(attempt, baseMs = 2_000, maxMs = 60_000) {
41    return Math.min(maxMs, baseMs * 2 ** Math.max(0, attempt - 1));
42  }
```

MongoDB-backed reads (notifications and activity feeds) are wrapped by `guarded`: if MongoDB is down or slower than 1.5 s, the request still succeeds with an empty result and `degraded: true`. Registration, which depends only on PostgreSQL, is unaffected (BE-I25) (Listing 2.22).

Listing 2.22: Graceful degradation for MongoDB reads (`backend/src/services/feedService.js`, lines 6–13)

```
 6  async function guarded(label, work, fallback) {
 7    if (mongoState() !== 'up') return { ...fallback, degraded: true };
 8    try {
 9      return { ...(await withTimeout(work(), mongoTimeoutMs, label)), degraded:
           false };
10    } catch {
11      return { ...fallback, degraded: true };
12    }
13  }
```

2.3.10 Event-driven design: bus, outbox dispatcher and subscribers (M3.12)

The bus extends Node's `EventEmitter`. A plain `emit()` is fire-and-forget, but the dispatcher must know whether delivery succeeded in order to retry, so `publish` waits for every listener and rejects if any of them failed (Listing 2.23).

Listing 2.23: EventEmitter-based event bus (`backend/src/events/eventBus.js`, lines 6–21)

```
 6  export class EventBus extends EventEmitter {
      // … 5 lines omitted …
12    async publish(type, payload) {
13      const listeners = this.listeners(type);
14      const results = await Promise.allSettled(listeners.map((listener) =>
            Promise.resolve().then(() => listener(payload))));
15      const failures = results.filter((r) => r.status === 'rejected').map((r) =>
            r.reason);
16      if (failures.length) {
17        throw new AggregateError(failures, `${failures.length} of
              ${listeners.length} subscribers failed for ${type}:
              ${failures[0]?.message}`);
18      }
19      return listeners.length;
20    }
21  }
```

The dispatcher delivers each outbox message with a timeout. On success the message is marked published; on failure it is rescheduled with exponential backoff, and the round stops so that a down MongoDB is not hammered (Listing 2.24).

Listing 2.24: Delivering one outbox message (`backend/src/events/outboxDispatcher.js`, lines 22–38)

```
22  async function deliver(message) {
23    try {
24      // The idempotency key travels with the message so subscribers can ignore
            re-deliveries.
25      const outboxId = message.messageId ?? String(message.id);
26      await withTimeout(bus.publish(message.type, { ...message.payload, outboxId
            }), deliveryTimeoutMs, `deliver ${message.type}`);
27      await outbox.markPublished(message.id);
28      stats.delivered++;
29      return true;
30    } catch (err) {
31      const attempts = message.attempts + 1;
32      await outbox.markFailed(message.id, err.message, new Date(clock().getTime()
            + backoffMs(attempts)));
33      stats.failed++;
34      stats.lastError = err.message;
35      logger.warn('Outbox delivery failed; will retry', { id: String(message.id),
            type: message.type, attempts, error: err.message });
36      return false;
37    }
38  }
```

Subscribers register themselves at start-up and know nothing about registrations, transactions or HTTP. The notification subscriber turns each past-tense event into a message for the affected student (Listing 2.25).

Listing 2.25: A subscriber reacting to past-tense events
(`backend/src/events/subscribers/notificationSubscriber.js`, lines 34–46)

```
34  export function registerNotificationSubscriber(bus, { store }) {
35    for (const [type, template] of Object.entries(TEMPLATES)) {
36      bus.on(type, (msg) =>
37        store.recordNotification({
38          outboxId: msg.outboxId,
39          userId: msg.userId,
40          ...template(msg),
41          // Expire 30 days after the fact happened (TTL index on expiresAt).
42          expiresAt: new Date(new Date(msg.at).getTime() + TTL_MS),
43        }),
44      );
45    }
46  }
```

2.3.11 Streams: the attendee CSV export (M2.2, M3.3)

The export reads attendees in batches from the database and pushes CSV lines through a `Readable` stream into the response. Memory use stays flat however many attendees there are, and `pipeline` waits whenever the client reads slowly. The header row is pulled first, so authorisation and 404 errors still produce a normal JSON error before any CSV byte is sent (Listing 2.26). Because organisers open the file in a spreadsheet, values starting with `=`, `+`, `-` or `@` are prefixed with a quote to prevent formula injection, a defensive-programming measure (BE-U38) (Listing 2.27).

Listing 2.26: Streaming CSV export with back-pressure
(`backend/src/controllers/registrations.controller.js`, lines 26–37)

```
26  exportCsv: asyncHandler(async (req, res) => {
27    const lines = registrationService.exportCsv(req.user, req.valid.params.id);
28    // Pull the header first: authorisation/404 errors surface before any byte is
          sent.
29    const first = await lines.next();
30    res.setHeader('Content-Type', 'text/csv; charset=utf-8');
31    res.setHeader('Content-Disposition', `attachment;
          filename="event-${req.valid.params.id}-attendees.csv"`);
32    async function* all() {
33      if (!first.done) yield first.value;
34      yield* lines;
35    }
36    await pipeline(Readable.from(all()), res);
37  }),
```

Listing 2.27: Safe CSV cells (RFC 4180 quoting, formula guard)
(`backend/src/services/registrationService.js`, lines 238–246)

```
238  function csvCell(value) {
239    let text = String(value ?? '');
240    if (/^[=+\-@]/.test(text)) text = `'${text}`;
241    return /[",\n\r]/.test(text) ? `"${text.replaceAll('"', '""')}"` : text;
242  }
243
244  export function csvLine(values) {
245    return values.map(csvCell).join(',') + '\n';
246  }
```

2.3.12 CRUD over REST: the tasks resource (M1.5, M1.13–M1.15, M2.5, M3.3)

Tasks are the plainest resource in the API: each HTTP method on `/tasks` or `/tasks/:id` maps to one create, read, update or delete operation, and the route table reads like the CRUD table in Section 2.6 (Listing 2.28). The same pipeline applies as everywhere else: identity, an express-validator chain (an empty title gives 400 “Title is required”), a thin controller and a service.

Listing 2.28: CRUD routes for tasks (`backend/src/routes/tasks.routes.js`, lines 7–16)

```
 7  export function createTasksRouter({ tasks, auth }) {
 8    const router = express.Router();
 9    router.use(auth.requireUser); // tasks are personal, so every route needs a
       known user
10    router.get('/', listTasksRules, validate, tasks.list);
11    router.post('/', createTaskRules, validate, tasks.create);
12    router.get('/:id', taskIdRules, validate, tasks.get);
13    router.patch('/:id', updateTaskRules, validate, tasks.update);
14    router.delete('/:id', taskIdRules, validate, tasks.remove);
15    return router;
16  }
```

The service enforces ownership. A task that belongs to someone else is reported exactly like a missing one (404), so its existence is not revealed (a trust boundary, M3.3). An empty update is rejected with 400, and `completedAt` is kept in step with `completed`, which a CHECK constraint enforces again in the database (Listing 2.29).

Listing 2.29: Ownership check and update rules for tasks (`backend/src/services/taskService.js`, lines 9–30)

```
 9  async function requireOwn(user, id) {
10    const task = await tasks.findForUser(id, user.id);
11    if (!task) throw new AppError(`Task ${id} not found`, 404, 'TASK_NOT_FOUND');
12    return task;
13  }
14
15  return {
      // … 6 lines omitted …
22    /** Edit fields and/or tick a task done; completedAt follows completed (a
       CHECK constraint enforces the same rule). */
23    async update(user, id, changes) {
24      const fields = EDITABLE.filter((key) => changes[key] !== undefined);
25      if (fields.length === 0) throw new AppError('Nothing to update: send title,
         notes, dueDate or completed', 400, 'EMPTY_UPDATE');
26      const task = await requireOwn(user, id);
27      const data = Object.fromEntries(fields.map((key) => [key, changes[key]]));
28      if (changes.completed !== undefined && changes.completed !== task.completed)
         data.completedAt = changes.completed ? clock() : null;
29      return tasks.update(id, data);
30    },
```

2.4 Connection with Full Stack Web Development and Advanced Database Systems

The API sits between the other two parts of SeatSure, and its design is shaped by both. Figure 2.2 follows one registration through all three layers.

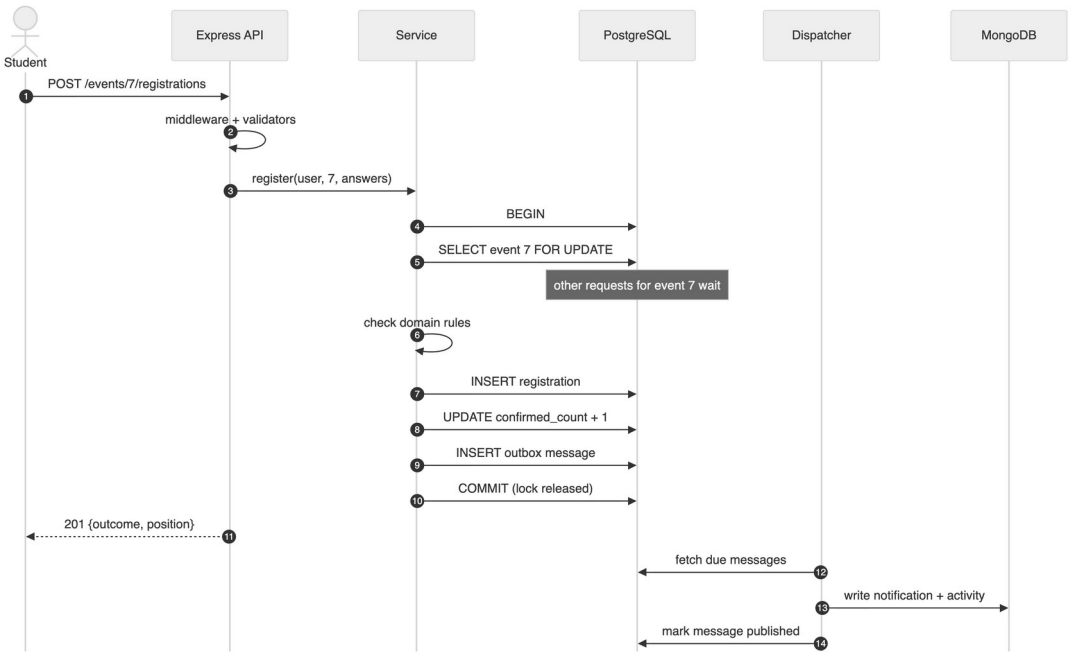


Figure 2.2: One registration across the client, the API and the two databases

- **Towards the client (Full Stack).** The React client relies on the response envelope and the error codes: `toApiError` reads `code`, `message`, `errors[]` and `requestId`, and React Hook Form shows each `errors[].field` (for example `answers.tshirt`) under the matching input. CORS allows the React origins and exposes the headers the client reads. Pagination metadata (`nextCursor`, `hasMore`, `totalPages`) drives infinite scroll and page controls.
- **Towards the databases (Advanced Database).** The services call repository functions in `@seatsure/db` and never see SQL. The repository behind `lockForUpdate` issues `SELECT … FOR UPDATE`, and `withTransaction` wraps Prisma's interactive `$transaction`. Database failures arrive as storage-neutral `DbError` kinds (`UNIQUE_VIOLATION`, `CHECK_VIOLATION`, `WRITE_CONFLICT`, `TIMEOUT`), which the error handler maps to 409, 503 and so on, and which the retry logic uses to decide whether an attempt is worth repeating.
- **Defence in depth.** The service decides seats under a lock, and PostgreSQL enforces the same rule a second time with `CHECK (confirmed_count <= capacity)` and a partial

unique index (one active registration per student). Even a bug in the service could not store an overbooked event.

- **Two databases, one truth.** The outbox table lives in PostgreSQL, so a registration and its "tell MongoDB" message commit together; the dispatcher and the subscribers deliver it to MongoDB afterwards. MongoDB can lag behind by about a second, but it can never miss or invent an event.

2.5 Setting Up and Running the Project

The project needs Node.js 22 and PostgreSQL 18. MongoDB is optional in development: when no MongoDB address is configured, an npm-distributed MongoDB server starts automatically. From the `seatsure` folder:

1. Install the dependencies of all three workspaces (`frontend`, `backend`, `db`): `npm install`.
2. Copy `.env.example` to `.env` and `frontend/.env.example` to `frontend/.env`, then set `DATABASE_URL` and `DATABASE_URL_TEST` to your PostgreSQL user.
3. Create, migrate and seed the databases: `npm run setup` (it runs `db:create`, `db:migrate` and `db:seed`). Alternatively, load `db/schema.sql` with `psql` and run `npm run db:seed`.
4. Start the backend and the frontend together: `npm run dev`. The API listens on http://localhost:4000 and the React app on http://localhost:5173.
5. Open http://localhost:5173 in a browser and choose a demo user in the header, for example Priya Singh (student) or Aarav Mehta (organiser).
6. Run the automated tests with `npm test`, and the live API checks with `npm run test:api` while the API is running.

If PostgreSQL cannot be installed, the README explains how to use free hosted databases instead (Neon or Supabase for PostgreSQL, MongoDB Atlas for MongoDB).

The complete API reference is Table 2.2 in Section 2.1. Every route is under `/api/v1` and answers with the same JSON envelope: `{status, data, meta}` on success and `{status, code, message, errors?, requestId}` on failure.

2.6 CRUD over REST: Route Mapping and Validation

SeatSure exposes its data as REST resources, so each create, read, update and delete operation is one HTTP method on one URL. Table 2.3 shows the mapping for its three resources (tasks, events and registrations), together with the database operation behind each route and the status

codes a client can expect. Tasks are the plainest example: a personal to-do list with no business rules beyond validation and ownership.

Table 2.3: CRUD operations mapped to REST routes (under `/api/v1`)

Operation	Route	Database operation	Success	Typical errors

Create	`POST /tasks`
	`POST /events`
	`POST /events/:id/registrations`
		`task.create`
		`event.create` (draft)
		INSERT inside a transaction that locks the event row
			201 + `Location`	400, 403, 409, 422

Read	`GET /tasks`, `GET /tasks/:id`
	`GET /events`, `GET /events/:id`
	`GET /me/registrations`
		`findMany` scoped to the user
		`findMany` with a cursor
		`findUnique` / `include` for related data
			200	400, 404

Update	`PATCH /tasks/:id`
	`PATCH /events/:id`
	`PATCH /registrations/:id`
		`task.update` (edit, tick done)
		`event.update` (edit, publish, cancel)
		`updateMany` for an idempotent check-in
			200	400, 403, 404, 409

Delete	`DELETE /tasks/:id`
	`DELETE /events/:id`
	`DELETE /registrations/:id`
		`task.delete`
		`event.delete` (drafts only)
		Status set to CANCELLED (soft delete) plus promotion
			204
			204
			200
				404, 409

Deleting a registration is a *soft* delete. The row is kept with status CANCELLED, so the history stays available for analytics, and the partial unique index ignores cancelled rows, so the student may register again. The response is 200 with a body rather than 204, because the client needs to know who was promoted into the freed seat.

Why validation runs on both the client and the server. The same input is checked in three places, each for a different reason:

- **In the browser:** React Hook Form checks required fields, lengths and cross-field rules (the end must come after the start; the capacity must fit the venue) before any request is sent. The user gets instant feedback and no request is wasted.
- **On the server:** the browser is under the user's control, and anyone can bypass the form with `curl` or the browser's developer tools. The API therefore validates every request again with express-validator. It answers 400 with a list of the invalid fields (live checks API-02 and API-10), drops fields a client may not set (mass assignment), and applies the business rules (409 or 422).
- **In the database:** CHECK, UNIQUE and FOREIGN KEY constraints protect the rules that must never break, such as `confirmed_count <= capacity`, even if a bug ever slips past the code above.

In short, client-side validation makes the form pleasant to use, while server-side and database validation make the system safe.

Chapter III – Results

3.1 Testing Approach

The API was tested at three levels:

- **Unit tests (38, `BE-U01`–`BE-U38`).** The domain rules are tested as pure functions, and the services are built with in-memory fake repositories, so they run without HTTP or a database. Middleware and the reliability utilities are tested in isolation.
- **HTTP integration tests (33, `BE-I01`–`BE-I33`).** Supertest sends real HTTP requests to the complete Express application, backed by a dedicated PostgreSQL test database and an in-memory MongoDB. These include the concurrency tests, in which 50 requests are fired at the same instant with `Promise.all`, and an outage test that disconnects MongoDB.
- **System-level checks.** Fourteen live API checks (`npm run test:api`) exercise every CRUD route and error case on the running server (Section 3.4). A live demo script fires simultaneous registrations at it, and a race experiment compares the workflow with and without the row lock.

Every test case is recorded with an ID, the concept and lesson it proves, the scenario, the input, and the expected and actual results (Appendix A). The raw output of the run is saved in `docs/test-results/backend-tests.txt`.

3.2 Test Results Summary

Table 3.1: Automated API tests by area (27 Oct 2026, `npm run test:backend`)

Area (lessons)	Test IDs	Tests	Result
Domain rules as pure functions (M3.2, M2.5, M2.6)	BE-U01 – U08	8	8 passed
Services with fake repositories: workflows, retries, authorisation (M3.2, M3.3, M3.6, M3.11, M3.12)	BE-U09 – U23	15	15 passed
Middleware, errors, reliability utilities, streams (M2.2, M2.4–M2.7, M3.11, M3.12)	BE-U24 – U38	15	15 passed
HTTP integration incl. concurrency and outage (M1.5–M3.12)	BE-I01 – I26	26	26 passed
Tasks: CRUD over REST over HTTP (M1.5, M1.13–M1.15, M2.5, M3.3)	BE-I27 – I33	7	7 passed
Total		**71**	**71 passed**

3.3 Results

- **All 71 automated tests passed** in 7.3 s on 27 October 2026.

- **CRUD over REST for tasks (BE-I27–I33):** create returned 201 with a `Location` header; an empty title returned 400 "Title is required" and stored nothing; deleting twice returned 204 then 404, as did `DELETE /tasks/99`; another user's task was invisible (404) and a smuggled `userId` was ignored; an empty update and an invalid date returned 400, and a request without a user 401.
- **No overbooking under load (BE-I21):** 50 students registering at the same instant for 3 seats all received 201; exactly 3 were confirmed, the other 47 received waitlist positions 1 to 47 with no gaps or repeats, and the stored counter was 3.
- **No duplicate processing (BE-I22):** one student double-clicking ten times produced one 201, nine 409 `ALREADY_REGISTERED` responses and a single row.
- **No lost update (BE-I23):** three simultaneous cancellations promoted three *different* students, the three oldest on the waitlist.
- **Mass assignment blocked (BE-I08, BE-I13):** smuggled `status`, `confirmedCount` and `createdById` fields were ignored; the event was created as a DRAFT with the caller as creator.
- **No leaks (BE-U29):** an unexpected `TypeError` produced a generic 500 response with no stack trace or type name.
- **Graceful degradation (BE-I25):** with MongoDB disconnected, registration still returned 201, feeds returned `degraded: true`, `/health` reported MongoDB as down, and the outbox kept the messages until MongoDB returned. This test found a real bug: after re-seeding, outbox ids restarted at 1 and were wrongly treated as duplicates by a persistent MongoDB. It was fixed with a UUID message id (decision D-11).

The same guarantee holds against the running server. Listing 3.1 shows the output of the live demonstration, in which 50 registrations are fired at a 10-seat event.

Listing 3.1: Live concurrency demo against the running API (`npm run demo:concurrency -- --seats 10 --students 50`)

```
SeatSure - live concurrency demo (2026-10-26 15:48 IST)
Command: npm run demo:concurrency -- --seats 10 --students 50   (API running on
    localhost:4000)

Event #31: 10 seats. Firing 50 registrations at the same moment...

┌─────────────────┬────────┐
│ (index)         │ Values │
├─────────────────┼────────┤
│ requests        │ 50     │
│ HTTP 201        │ 50     │
│ confirmed       │ 10     │
│ waitlisted      │ 40     │
│ errors          │ 0      │
│ wall time (ms)  │ 414    │
└─────────────────┴────────┘
Waitlist positions handed out: 1..40 (40 unique)
Database says: confirmedCount=10/10, waitlist=40
Seat counter mismatches across all events: 0

No overbooking: exactly 10 confirmed, everyone else waitlisted in order.
```

To show that the lock is what makes the difference, a race experiment ran a simplified version of the workflow (read the count, pause for 20 ms, insert; no duplicate check or outbox) with and without `SELECT … FOR UPDATE` on a 3-seat event (Figure 3.1, Table 3.2). Without the lock, the event was overbooked as soon as more than three requests arrived together; the overbooking stopped growing at 10 only because the connection pool allows ten transactions at a time. With the lock, the result was exactly 3 every time.

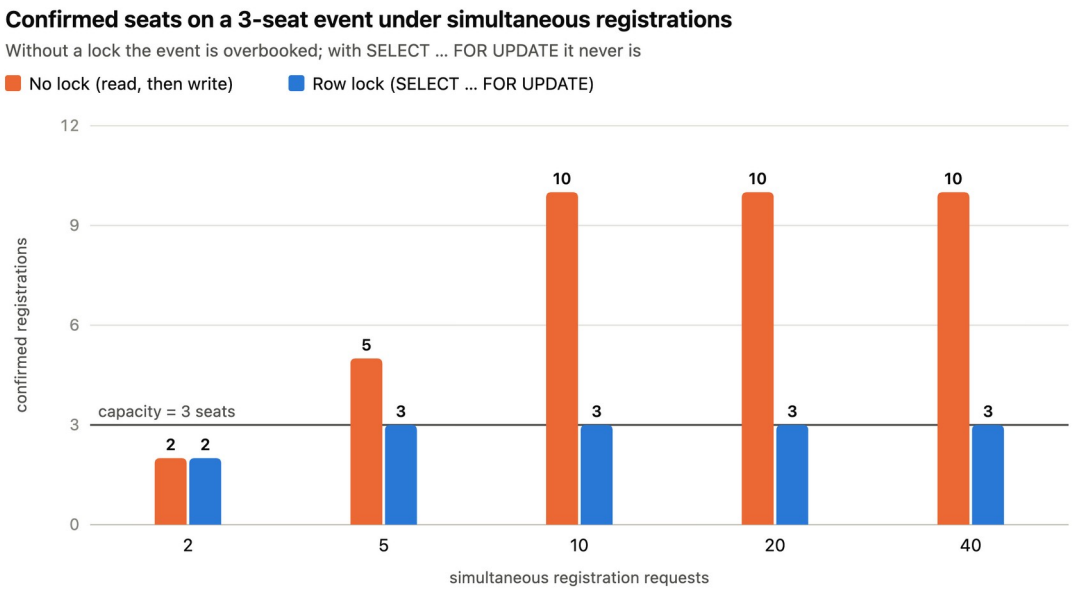


Figure 3.1: Confirmed seats on a 3-seat event under simultaneous registrations, with and without the row lock

Table 3.2: Race experiment (`npm run experiment:race -w @seatsure/db`)

Simultaneous requests	Confirmed, no lock	Confirmed, FOR UPDATE	Time, no lock	Time, FOR UPDATE
2	2	2	42 ms	54 ms
5	**5** (overbooked)	3	33 ms	119 ms
10	**10** (overbooked)	3	42 ms	246 ms
20	**10** (overbooked)	3	69 ms	489 ms
40	**10** (overbooked)	3	124 ms	964 ms

The screenshots below show the same behaviour from the user's side. Figure 3.2 shows the four task operations through the interface, each handled by one route of Table 2.3.

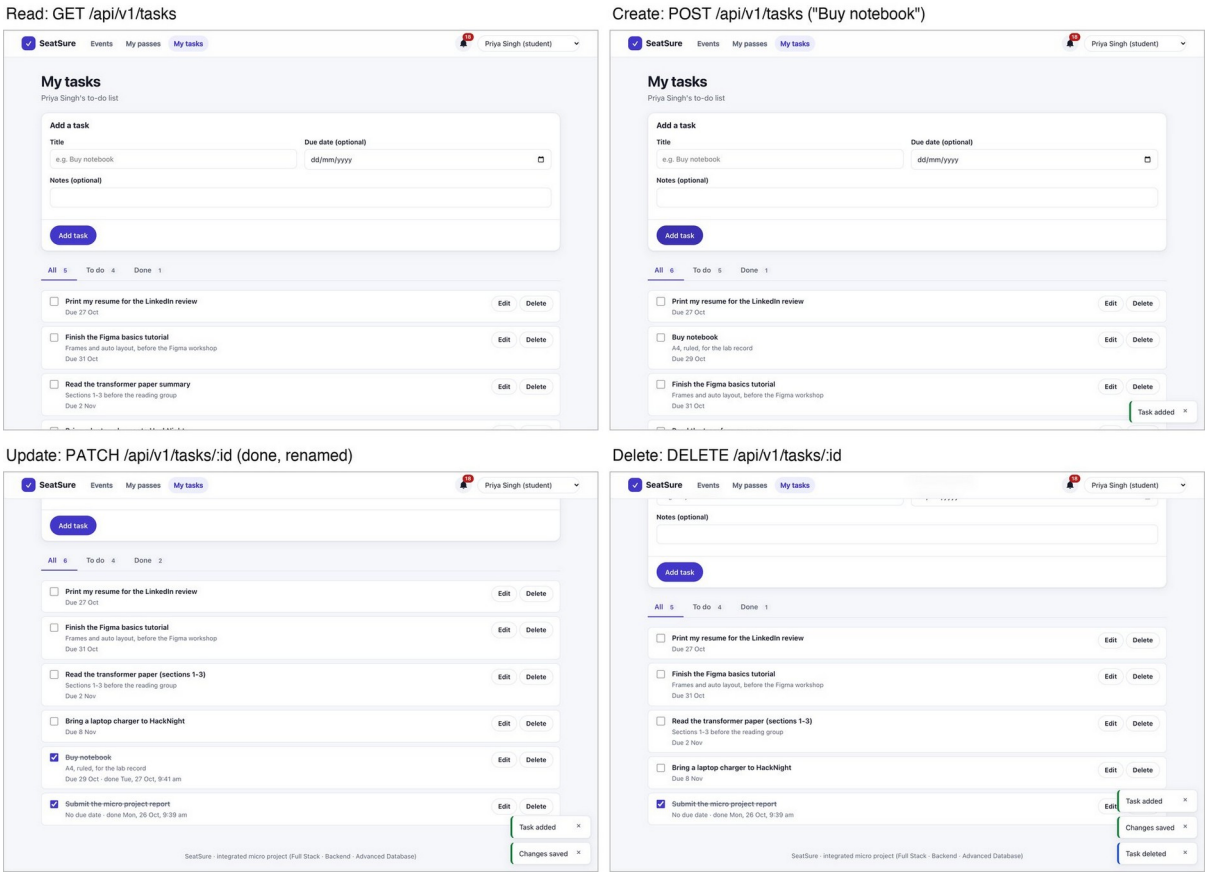


Figure 3.2: Tasks: read, create, update and delete through the user interface, each handled by one REST route

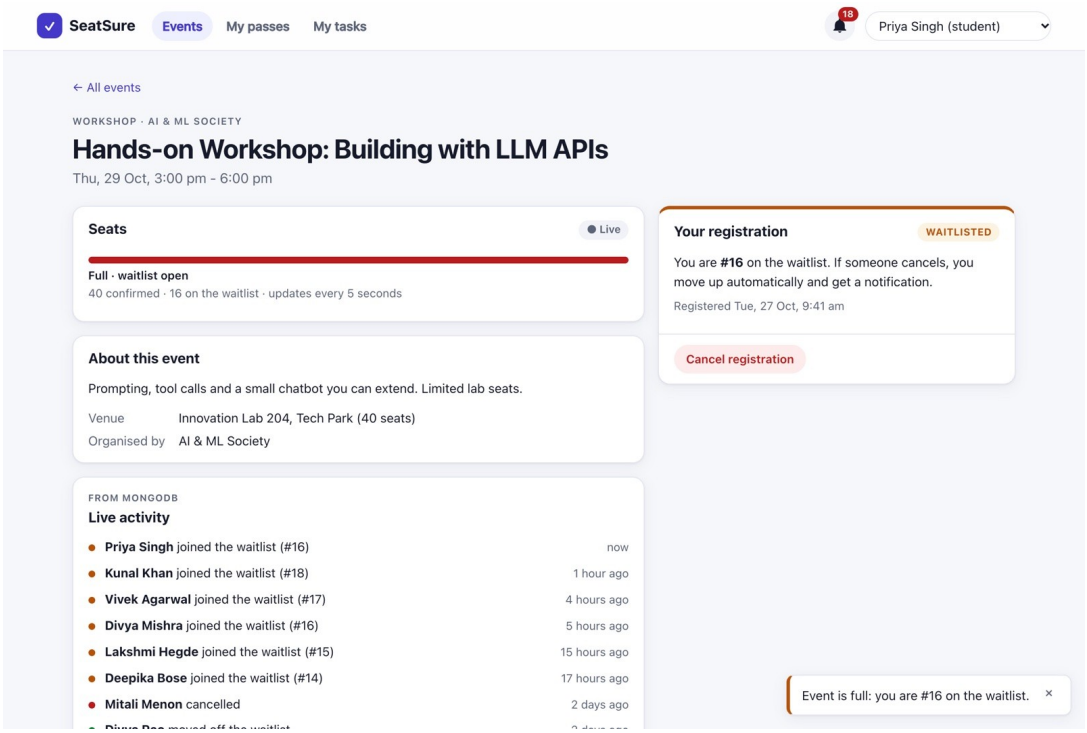


Figure 3.3: A registration for a full event: the API answers 201 with outcome WAITLISTED and the position

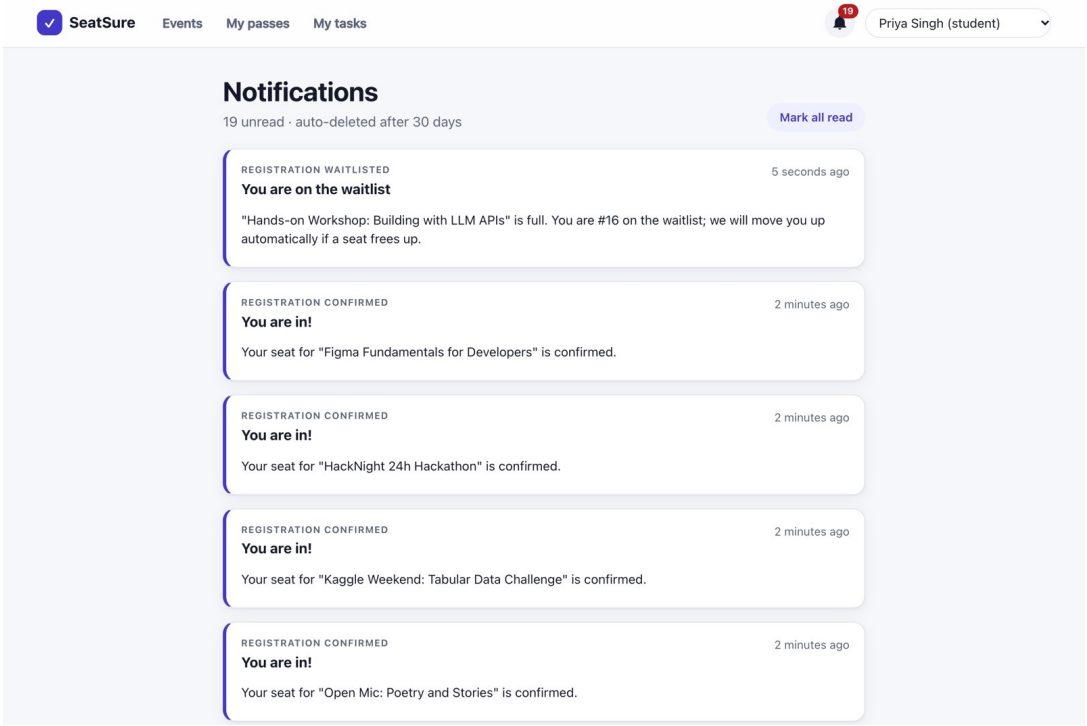


Figure 3.4: Notifications written to MongoDB by event subscribers through the outbox

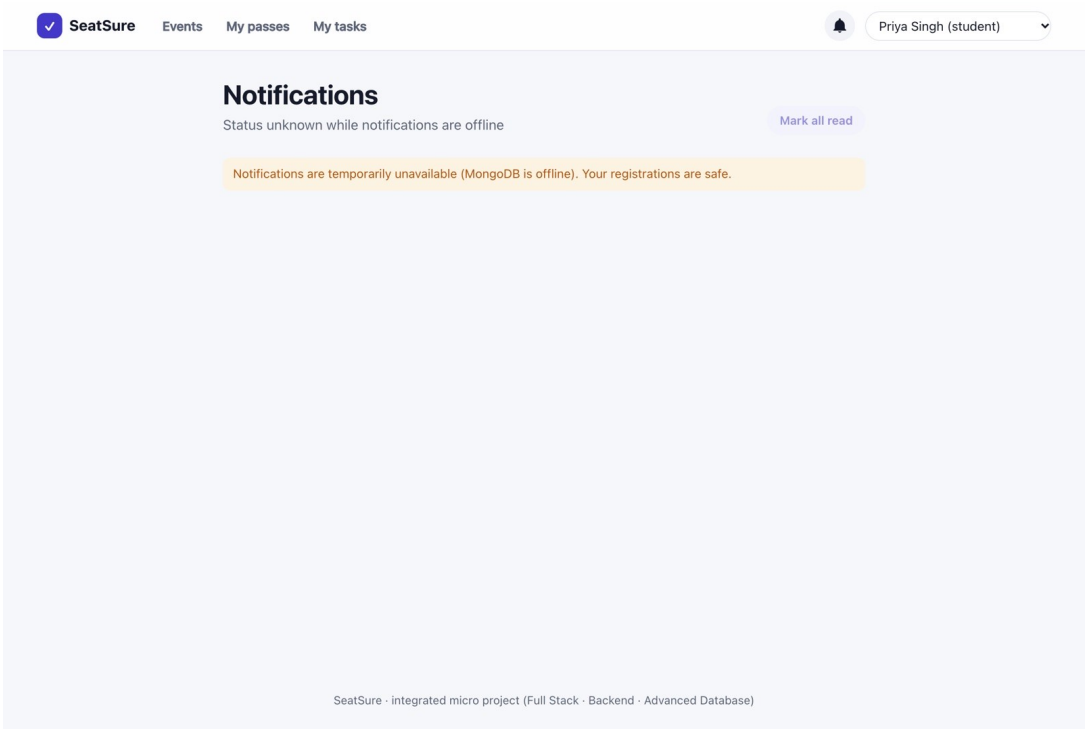


Figure 3.5: Simulated MongoDB outage: the notifications page reports that the feed is temporarily unavailable

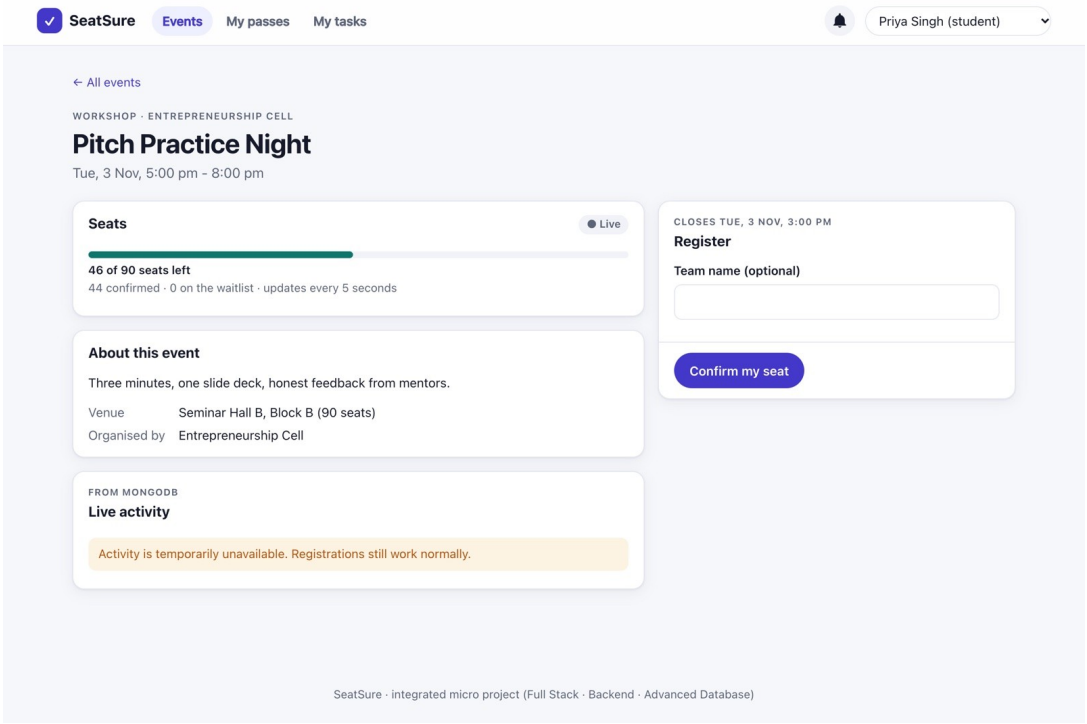


Figure 3.6: During the same outage, registration still works because it depends only on PostgreSQL

3.4 Live API Test Cases

Besides the automated suites, 21 real HTTP requests were sent to the running API on 27 October 2026 with `npm run test:api`, which records each expected and actual result (raw output in `docs/test-results/api-checks.txt`). API-01 to API-03 are the example test

cases from the course guidelines (create a task, an empty title and a missing id); the rest cover every other CRUD operation and the main error cases. **21 of 21 behaved as expected** (Table 3.3).

Table 3.3: Live API test cases: expected and actual results (paths relative to `/api/v1`)

Test	Operation	Request	Expected	Actual	Result
API-01	Create	`POST /tasks {title:"Buy notebook"}`	201 new task	201 id 48, completed false, Location /api/v1/tasks/48	Pass
API-02	Invalid input	`POST /tasks {title:""}`	400 error	400 VALIDATION_ ERROR: Title is required	Pass
API-03	Missing ID	`DELETE /tasks/99`	404 not found	404 TASK_NOT_FOUND	Pass
API-04	Read	`GET /tasks`	200, my tasks incl. "Buy notebook"	200 6 tasks (5 to do, 1 done)	Pass
API-05	Update (toggle)	`PATCH /tasks/48 {completed:true}`	200, done with completedAt	200 completed true, completedAt set	Pass
API-06	Update (edit)	`PATCH /tasks/48 {title:"Buy two notebooks"}`	200, new title	200 "Buy two notebooks"	Pass
API-07	Delete	`DELETE /tasks/48`	204 No Content	204 No Content	Pass
API-08	Read (events)	`GET /events?limit=3`	200, 3 events + nextCursor	200 3 events, nextCursor 4	Pass
API-09	Create	`POST /events {title:"API check: Git workshop", …}`	201 + Location, status DRAFT	201 DRAFT, Location /api/v1/events/32	Pass
API-10	Invalid input	`POST /events {title:"", capacity:0, …}`	400 VALIDATION_ ERROR for title and capacity	400 VALIDATION_ ERROR (title, capacity)	Pass
API-11	Read (one)	`GET /events/32`	200, the new draft	200 "API check: Git workshop" (DRAFT)	Pass
API-12	Update	`PATCH /events/32 {title:"… (updated)"}`	200, new title	200 "API check: Git workshop (updated)"	Pass
API-13	Delete	`DELETE /events/32`	204 No Content	204 No Content	Pass
API-14	Missing ID	`DELETE /events/32 (again)`	404 not found	404 EVENT_NOT_FOUND	Pass
API-15	Update (workflow)	`PATCH /events/33 {status:"PUBLISHED"}`	200, PUBLISHED	200 PUBLISHED	Pass
API-16	Create (registration)	`POST /events/33/registrations {answers:{}}`	201, CONFIRMED	201 CONFIRMED	Pass
API-17	Duplicate	Same request again	409 ALREADY_ REGISTERED	409 ALREADY_ REGISTERED	Pass
API-18	Missing user	Same request without X-User-Id	401	401 UNAUTHENTICATED	Pass
API-19	Delete (cancel)	`DELETE /registrations/2517`	200, cancelled	200 previous status CONFIRMED	Pass
API-20	Missing ID	`DELETE /registrations/99999`	404 not found	404 REGISTRATION_ NOT_FOUND	Pass
API-21	Unknown route	`GET /does-not-exist`	404 ROUTE_NOT_ FOUND	404 ROUTE_NOT_FOUND	Pass

3.5 Discussion

The lock has a cost, and it is local. With the lock, registrations for *one* event run one after another. In the race experiment, where every transaction deliberately pauses for 20 ms, 40 simultaneous requests took 964 ms instead of 124 ms; without that artificial pause, the real API

served 50 simultaneous registrations in 414 ms. Because each event has its own row lock, registrations for different events still run in parallel. For a campus system, where even a popular event receives a few hundred registrations in its first minute, this is a good trade: correctness is guaranteed and the wait stays well under a second.

Why a row lock rather than SERIALIZABLE? SERIALIZABLE isolation would also prevent overbooking, but every conflict would surface as an error that must be retried, and under a burst most attempts would fail and retry. A row lock makes requests queue instead, so almost all of them succeed on the first attempt; retries remain only as a safety net for deadlocks and pool timeouts.

Events decouple, the outbox makes them safe. The registration code does not know that notifications exist. Writing the message in the same transaction as the registration, instead of calling MongoDB directly, means that a crash or a MongoDB outage can delay a notification but never lose one, as BE-I25 demonstrates.

Chapter IV – Conclusion

This report presented the Back-End Web Development part of SeatSure. The Express 5 API applies the concepts of all three modules of the course. Module 1 provided the REST resources, a uniform JSON contract and correct status codes. Module 2 provided the architecture: an ordered middleware pipeline, request validation with mass-assignment protection, a central error-handling pipeline, a CSV export streamed with back-pressure, and a layered design in which pure business rules are tested with fake repositories. Module 3 provided the engineering: explicit domain workflows, protection against race conditions through transactions with row locks, cursor and offset pagination, API versioning with deprecation, reliability patterns (retries with backoff and jitter, timeouts and graceful degradation) and an event-driven design built on an EventEmitter bus and a transactional outbox.

All 71 automated tests pass. Fifty simultaneous registrations for three seats produce exactly three confirmations, simultaneous cancellations promote different students, and a MongoDB outage costs only the notification feed, never a registration. The main lesson of the project is that concurrency bugs cannot be found by clicking through an application: they have to be reproduced on purpose, as the race experiment does, and then prevented by design.

4.1 Limitations

- Identity is a placeholder (`X-User-Id`); there is no real authentication or authorisation token yet. This is intentional, because authentication is taught in Module 4.
- There is no rate limiting, so one client could flood the API with requests.
- The event bus and the outbox dispatcher run inside the API process. Several API instances would need a shared message broker, or `SELECT … FOR UPDATE SKIP LOCKED` so that dispatchers do not claim the same messages.
- The per-event lock serialises registrations for one very popular event; at much larger scale, a queue in front of hot events would smooth the load.

4.2 Future Scope

- Replace the demo identity with session- or JWT-based authentication (Module 4).
- Add rate limiting per user and per IP address.
- Move event delivery to a message broker so that the API can scale to several instances.

- Publish an OpenAPI description of the API and generate the client's service layer from it.

References

1. Kalvium, *Back-End Web Development* livebook, Semester 3: Module 1 (Backend Development Foundations), Module 2 (Backend Application Architecture), Module 3 (Backend Engineering Design and Principles). Accessed 26 October 2026.
2. Express 5 documentation: routing, writing middleware, error handling. https://expressjs.com
3. express-validator documentation: validation chains, `validationResult`, `matchedData`. https://express-validator.github.io/docs
4. Node.js documentation: `events` (EventEmitter) and `stream` (`Readable.from`, `pipeline`). https://nodejs.org/docs/latest-v22.x/api
5. R. Fielding, M. Nottingham, J. Reschke, *HTTP Semantics*, RFC 9110, IETF, 2022.
6. Y. Shafranovich, *Common Format and MIME Type for Comma-Separated Values (CSV) Files*, RFC 4180, IETF, 2005.
7. E. Wilde, *The Sunset HTTP Header Field*, RFC 8594, IETF, 2019; and *The Deprecation HTTP Response Header Field*, RFC 9745, IETF, 2025.
8. PostgreSQL 18 documentation: explicit locking (`SELECT … FOR UPDATE`) and transaction isolation. https://www.postgresql.org/docs/18/
9. C. Richardson, “Pattern: Transactional outbox”, microservices.io.
10. SeatSure project documentation: `README.md` (API reference, traceability matrix), `DECISIONS.md` and `docs/testing/test-cases-backend.md`.

Appendix A – Test Cases

Automated tests: `npm run test:backend` (Vitest and Supertest), 27 October 2026. Result: **71 / 71 passed**. Raw output: `docs/test-results/backend-tests.txt`. Unit tests (`BE-U`) use in-memory fake repositories; integration tests (`BE-I`) send real HTTP requests to the Express app backed by PostgreSQL and MongoDB.

Table A.1: Domain rules and workflows (unit)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
BE-U01	M3.2 workflow (state machine)	Event status transitions	DRAFT→PUBLISHED, PUBLISHED→CANCELLED, DRAFT→CANCELLED, CANCELLED→PUBLISHED	First two allowed, last two `409 INVALID_STATUS_TRANSITION`	As expected	Pass
BE-U02	M3.2 domain rules	Registering for a draft or a closed event	status DRAFT; closes-at in the past	`409 REGISTRATION_NOT_OPEN` / `409 REGISTRATION_CLOSED`	As expected	Pass
BE-U03	M3.2 domain rules	Check-in window	Before, during, after the window	NOT_OPEN / ok / CLOSED	As expected	Pass
BE-U04	M3.2 seat decision	Seat or waitlist	1/2 and 2/2 confirmed	true / false	As expected	Pass
BE-U05	M3.2 capacity rules	Shrinking below confirmed; exceeding the venue	30 confirmed → 20; 150 > venue 100	409 CAPACITY_BELOW_CONFIRMED; 422 CAPACITY_EXCEEDS_VENUE	As expected	Pass
BE-U06	M2.6 sanitisation	Answers are cleaned	`" Byte Busters "`, `"true"`	trimmed, boolean `true`	As expected	Pass
BE-U07	M2.5 validation (all errors at once)	Several bad answers	Unknown key, wrong option, missing required	422 with 3 field paths	`answers.hacker`, `answers.laptop`, `answers.tshirt`	Pass
BE-U08	M3.2 rules	Organiser's question list	Duplicate ids; choice with 1 option	422 INVALID_QUESTIONS	As expected	Pass

Table A.2: Services with fake repositories (unit)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
BE-U09	M3.2 workflow, M2.10 service	4 students, 2 seats	Sequential registrations	CONFIRMED, CONFIRMED, WAITLISTED #1, #2	As expected; counter = 2	Pass
BE-U10	M3.12 events via outbox	Outbox message per registration	2 registrations	`registration.confirmed`, `registration.waitlisted` (position 1); delivery requested twice	As expected	Pass
BE-U11	M3.2 rule	Same student twice	2 registrations	409 ALREADY_REGISTERED	As expected	Pass
BE-U12	M3.2 + M2.5	Answers checked against the event's own questions	Missing required select	422 INVALID_ANSWERS, then success with a valid answer	As expected	Pass
BE-U13	M3.11 retry	Transaction loses a write conflict once	Fake transaction throws `WRITE_CONFLICT` first	Retried, then succeeds (2 calls)	2 calls, CONFIRMED	Pass
BE-U14	M3.11 retry only when transient	Business-rule failure	DRAFT event	Not retried (1 call)	1 call	Pass
BE-U15	M3.2 workflow + M3.6 transaction	Cancel a confirmed seat	1 seat, 2 waitlisted	Oldest waitlisted promoted; counter unchanged; cancelled + promoted messages	As expected	Pass
BE-U16	M3.2	Cancel with nobody waiting	2 seats, 1 registered	Seat freed (counter 0), `promoted: null`	As expected	Pass
BE-U17	M3.3 trust boundaries	Who may cancel	Stranger / club organiser / owner again	403 / ok / 409 ALREADY_CANCELLED	As expected	Pass
BE-U18	M3.2	Cancel after the start	`startsAt` in the past	409 EVENT_STARTED	As expected	Pass
BE-U19	M3.2 idempotency	Check-in twice; check-in by a student	Organiser ×2, student ×1	403; `alreadyCheckedIn` false then true	As expected	Pass
BE-U20	M3.3 authorisation	Create an event	Student; organiser of another club; own club	403, 403, 201-equivalent DRAFT	As expected	Pass
BE-U21	M3.2	Capacity above the venue	80 > 50	422 with `capacity` field error	As expected	Pass
BE-U22	M3.2 + M3.6	Raise capacity with a waitlist	1 → 3 seats, 3 waitlisted	2 promoted, counter 3, capacity_changed + 2 promoted messages	As expected	Pass
BE-U23	M3.2 workflow	Publish, publish again, delete published	Same event	publishedAt set; second publish is a no-op; delete 409 NOT_A_DRAFT	As expected	Pass

Table A.3: Middleware, errors and reliability (unit)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
BE-U24	M2.4 request-id + timing middleware	Any request	`GET /ping`	`X-Request-Id` (UUID) matches `req.id`; `X-Response-Time` ends in "ms"	As expected	Pass
BE-U25	M3.3 trust boundary	Incoming request id	`trace-12345678` vs `<script>`	Reused vs replaced	As expected	Pass
BE-U26	M2.5 validation, M2.6 mass assignment	Bad body; extra field	`{title:'x', qty:'many'}`; `{…, isAdmin:true}`	400 listing `title`, `qty`; handler sees only declared fields	As expected	Pass
BE-U27	M2.7 AppError → envelope	Thrown AppError 404	`EVENT_NOT_FOUND`	`{status:'fail', code, message, requestId}`	Exact body	Pass
BE-U28	M2.7 + DB error mapping	DbError kinds	UNIQUE / NOT_FOUND / UNAVAILABLE	409 ALREADY_REGISTERED / 404 / 503	As expected	Pass
BE-U29	M2.7 programmer errors, M1.15 no leaks	`TypeError` thrown	—	500 generic message, no stack or type names	As expected	Pass
BE-U30	M2.6 payload limit, M1.9 catch-all	Malformed JSON; 5 KB body on a 1 KB limit; unknown route	—	400 MALFORMED_JSON; 413; 404 ROUTE_NOT_FOUND	As expected	Pass
BE-U31	M3.11 retry with backoff	Fails twice, then succeeds	—	3 attempts; delays grow	As expected	Pass
BE-U32	M3.11 retry budget	Always fails; permanent error	retries 2; shouldRetry false	3 calls then throw; 1 call	As expected	Pass
BE-U33	M3.11 timeout, backoff cap	200 ms task with a 20 ms timeout; backoff(1,2,3,10)	—	TimeoutError; 2 s, 4 s, 8 s, 60 s	As expected	Pass
BE-U34	M3.12 EventEmitter bus	2 subscribers; failing subscriber	—	Both called, returns 2; rejects "1 of 1 subscribers failed"	As expected	Pass
BE-U35	M3.12 + outbox	Two due messages	—	Delivered in order, both marked published, `outboxId` attached	As expected	Pass
BE-U36	M3.11 backoff	Subscriber failing (MongoDB down)	attempts = 2	Rescheduled 8 s later; round stops	+8000 ms	Pass
BE-U37	M3.11 timeout	Subscriber never settles	30 ms timeout	Marked failed: "timed out"	As expected	Pass
BE-U38	M2.2 streams (CSV rows)	Commas, quotes, formula injection	`a,b`, `say "hi"`, `=HYPERLINK(…)`	RFC 4180 quoting; formula prefixed with `'`	As expected	Pass

Table A.4: HTTP integration (Supertest → Express → PostgreSQL + MongoDB)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
BE-I01	M1.6 contract, M3.11 health	Health check	`GET /api/v1/health`	200; postgres up, mongo up, 0 counter mismatches	As expected	Pass
BE-I02	M1.15 envelope, M2.4 request id	Unknown route	`GET /api/v1/does-not-exist`	404 `ROUTE_NOT_FOUND` with the same request id as the header	As expected	Pass
BE-I03	M3.10 API versioning	Legacy unversioned path	`GET /api/events`	200 + `Deprecation: true` + `Link` to `/api/v1/events`	As expected	Pass
BE-I04	M3.9 cursor pagination, M3.8 metadata	Catalogue page 1 and 2	`limit=2`, then `cursor=…`	2 + 2 events in date order; `hasMore` true then false	As expected	Pass
BE-I05	M1.11 query params, M2.5 validation	Filter; invalid params	`category=TALK`; `limit=500&category=PARTY`	1 talk; 400 listing `category`, `limit`	As expected	Pass
BE-I06	M3.3 authorisation	Draft visibility	Student vs club organiser	404 vs 200 `canManage: true`	As expected	Pass
BE-I07	M1.5 REST, M1.15 status codes	Create an event	Student; organiser	403; 201 + `Location: /api/v1/events/:id`, DRAFT	As expected	Pass
BE-I08	M2.5 validation, M2.6 mass assignment	Invalid body; smuggled fields	`title:'Hi', capacity:0; status, confirmedCount:999, createdById`	400 listing 3 fields; 201 DRAFT, count 0, creator = caller	As expected	Pass
BE-I09	M3.2 rule over HTTP	Capacity above the venue	100 > 40	422 with `capacity` field error	As expected	Pass
BE-I10	M3.2 workflow	Publish; invalid target; delete published	`PATCH {status}`; `DELETE`	200 PUBLISHED; 400; 409 NOT_A_DRAFT	As expected	Pass
BE-I11	M3.3 authorisation	Another club's organiser edits	`PATCH title`	403 NOT_CLUB_ORGANISER	As expected	Pass
BE-I12	M2.4 auth placeholder, M3.3 trust boundary	Missing, malformed, unknown user header	none / `abc` / `99999`	401 / 400 / 401 UNKNOWN_USER	As expected	Pass
BE-I13	M3.2 workflow, M2.6	5 students, 3 seats; client sends `status: CONFIRMED`	`POST …/registrations` ×5	C, C, C, W#1, W#2 (injected status ignored)	As expected	Pass
BE-I14	M3.2 + M2.5	Duplicate; bad answers	Same student; `{tshirt:'XXXL', extra}`	409 ALREADY_REGISTERED; 422 listing `answers.extra`, `answers.tshirt`	As expected	Pass
BE-I15	M3.2 + M3.6	Cancel a confirmed seat with a waitlist	`DELETE /registrations/:id`	200, first waitlisted promoted; #2 becomes #1; second cancel 409	As expected	Pass
BE-I16	M3.2 idempotency	Check-in today vs in 3 days	`PATCH {checkedIn:true}`	First ok, second `alreadyCheckedIn`, early 409 CHECK_IN_NOT_OPEN	As expected	Pass

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
BE-I17	M3.9 offset pagination + JSONB filter	Attendees with `answers[tshirt]=M`, page size 2	Organiser; student	200, total 3, 2 pages, `checkInOpen:false`; student 403	As expected	Pass
BE-I18	M2.2 streams	CSV export	`GET …/registrations/export`	`text/csv`, header row + 4 rows	As expected	Pass
BE-I19	M3.3 + analytics	Analytics as student / organiser	`GET /analytics/overview`	403 / 200 with rank 1 = "Git Bootcamp", 21 daily rows	As expected	Pass
BE-I20	M3.3 single source of truth	Counter after mixed register/cancel	6 registrations, 2 cancels	`confirmed_count` = real CONFIRMED rows = 3	As expected	Pass
BE-I21	M3.4/M3.5 race condition, M3.6 transaction	**50 simultaneous registrations, 3 seats**	`Promise.all` of 50 POSTs	All 201; exactly 3 confirmed; positions 1…47 unique; counter 3	As expected	Pass
BE-I22	M3.5 duplicate processing	One student double-clicks 10×	10 parallel POSTs, same user	Exactly one 201, nine 409 ALREADY_REGISTERED, 1 row	As expected	Pass
BE-I23	M3.5 lost update	3 simultaneous cancellations	3 parallel DELETEs	3 different students promoted (the 3 oldest), counter 3	As expected	Pass
BE-I24	M3.12 event-driven + outbox	Registration reaches MongoDB	Register, drain outbox	Notification `REGISTRATION_CONFIRMED` + feed item `REGISTERED`; 0 pending	As expected	Pass
BE-I25	M3.11 graceful degradation	MongoDB outage, then recovery	Disconnect, register, read, reconnect, drain	Register 201; reads `degraded:true`; health mongo down; outbox keeps messages; delivered after recovery	As expected (this test found the outbox-id bug, DECISIONS.md D-11)	Pass
BE-I26	M3.12	Promotion notification	Cancel a confirmed seat	Promoted student gets `PROMOTED_FROM_WAITLIST`	As expected	Pass

Table A.5: Tasks: CRUD over REST (integration)

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
BE-I27	M1.13 create, M1.15 status codes	Create a task	`POST /api/v1/tasks {title:"Buy notebook"}`	201 + `Location: /api/v1/tasks/:id`, stored open	As expected	Pass
BE-I28	M2.5 validation	Empty title	`POST /api/v1/tasks {title:""}`	400 `VALIDATION_ERROR`, field `title` ("Title is required"); nothing stored	As expected	Pass

ID	Concept	Scenario	Input	Expected	Actual	Pass/Fail
BE-I29	M1.13 read, M3.3 trust boundary	List my tasks	2 own tasks (1 done) + 1 of another user	200, only my 2 tasks, open first; `meta {total 2, open 1, done 1}`	As expected	Pass
BE-I30	M1.14 update	Edit and tick done, then reopen	`PATCH {title, completed:true}`, then `{completed:false}`	200; `completedAt` set, then cleared	As expected	Pass
BE-I31	M1.14 delete, M1.15	Delete twice; missing id	`DELETE /api/v1/tasks/:id` ×2; `DELETE /api/v1/tasks/99`	204, then 404 `TASK_NOT_FOUND`; 404	As expected	Pass
BE-I32	M3.3 trust boundary, M2.6 mass assignment	Another user's task; smuggled `userId`	GET/PATCH/DELETE as another student	404 for all three; stored owner = caller	As expected	Pass
BE-I33	M2.5 validation, M2.4 auth placeholder	Empty update; bad date; no user	`PATCH {}`; `dueDate:"next friday"`; no `X-User-Id`	400 `EMPTY_UPDATE`; 400 field `dueDate`; 401	As expected	Pass

Table A.6: Live system demo, not counted in the 71 (`docs/test-results/concurrency-demo.txt`)

Requests	HTTP 201	Confirmed	Waitlisted	Errors	Waitlist positions	Counter mismatches	Wall time
50	50	**10**	**40**	0	1..40, all unique	0	414 ms

Appendix B – Concept Traceability

Every concept is tagged in the code as `// [BACKEND Mx.y: Concept]`. This table, taken from the project README, links each concept to the files that implement it, a way to demonstrate it, and the tests that prove it.

Table B.1: Traceability matrix: concept → code → test (Back-End Web Development)

Module	Concept	Files / functions	How to demo	Test IDs
M1.5	REST resource design	`routes/*.routes.js` (nouns, sub-resources `/events/:id/registrations`)	API table (Table 2.2)	BE-I04, BE-I07
M1.6 / M1.15	JSON contract, status codes, error envelope	`utils/respond.js` (`ok`, `created`, `noContent`); `middleware/errorHandler.js`	`curl -i`: 201 + Location, 204, 404, 409, 422	BE-U27, BE-I02, BE-I07
M1.9 / M1.11	Express server, Router modules, params vs query, catch-all 404	`server.js` (`app.listen`), `routes/index.js`, `middleware/notFound.js`	`GET /api/v1/nope`	BE-U30, BE-I02
M2.2	Streams	`controllers/registrations.controller.js` (`exportCsv`: `Readable.from` + `pipeline`); `registrationRepository.iterateForExport`	Attendees → Export CSV	BE-I18, BE-U38
M2.4	Middleware pipeline (request-id, timing, logging, order)	`app.js`; `middleware/requestId.js`, `middleware/requestTimer.js`	Response headers `X-Request-Id`, `X-Response-Time`; server log	BE-U24, BE-U25
M2.5	express-validator	`validators/*.validators.js`; `middleware/validate.js`	POST invalid event: 400 field list	BE-U26, BE-I05, BE-I08
M2.6	Hardening: sanitisation, mass assignment, payload limit	`validate.js` (`matchedData` allowlist); `express.json({limit:'10kb'})`; `eventRules.normaliseAnswers`	Send `status:"CONFIRMED"` → ignored; 20 KB body → 413	BE-I08, BE-I13, BE-U30
M2.7	AppError + asyncHandler + central handler	`utils/AppError.js`, `utils/asyncHandler.js`, `middleware/errorHandler.js`	Any 4xx/5xx shares one shape	BE-U27–U29
M2.9–2.11	Layered architecture, thin controllers, services, repository pattern	`routes/` → `controllers/` → `services/` → `@seatsure/db` repositories; `container.js` (composition root)	Unit tests swap in fake repositories	BE-U09–U23
M2.12	Supporting layers: config	`config/index.js` (env-driven, frozen)	—	—
M3.1 / M3.3	Data flow, trust boundaries, single source of truth	`middleware/auth.js` (validates `X-User-Id`); PostgreSQL authoritative, MongoDB derived	`/health` `seatCounterMismatches: 0`	BE-I12, BE-I20
M3.2	Domain rules & workflows	`services/rules/eventRules.js` (state machine, windows, answers); `registrationService.register/cancel/checkIn`; `eventService.update` (capacity → promotion)	Cancel → next person promoted	BE-U01–U23, BE-I10, BE-I15

Module	Concept	Files / functions	How to demo	Test IDs
M3.4–3.6	Concurrency, race conditions, transactions	`registrationService.register` (`transactWithRetry` → `lockForUpdate` → insert → counter → outbox)	`npm run demo:concurrency`	BE-I21–I23, DB-T04/T05
M3.8–3.9	API design: pagination, filtering, sorting metadata	`events.controller.js` (`meta.nextCursor/hasMore`, `meta.page/total/totalPages`)	Catalogue / attendees	BE-I04, BE-I17
M3.10	API versioning + deprecation	`routes/index.js` (`/api/v1`, `/api/events` alias); `middleware/deprecation.js`	`curl -i /api/events`	BE-I03
M3.11	Reliability: retries, timeouts, backoff, graceful degradation	`utils/retry.js`; `events/outboxDispatcher.js`; `services/feedService.js`, `analyticsService.activity`	Analytics → "Simulate MongoDB outage"	BE-U13, BE-U31–U37, BE-I25
M3.12	Event-driven (EventEmitter bus)	`events/eventBus.js` (`publish`); `events/subscribers/*`; dispatcher	Register → notification appears	BE-U34, BE-I24, BE-I26
M1.5, M1.13–1.15	CRUD over REST: the tasks resource	`routes/tasks.routes.js`, `validators/tasks.validators.js`, `controllers/tasks.controller.js`, `services/taskService.js`	`npm run test:api` (API-01–07)	BE-I27–I33